MatruRaksha — Complete Technical Reconstruction Document

AI-Powered Maternal Healthcare System for Rural India

> **Purpose**: This document is a complete, ground-up technical reference for the MatruRaksha system. Anyone reading this document — human or AI agent — should be able to recreate the entire project from scratch without any external resource.

Table of Contents

1. Project Overview & Problem Statement
2. System Architecture
3. Technology Stack & Versions
4. Directory Structure (Complete)
5. Environment Configuration
6. Database Schema (MongoDB)
7. Application Factory & Bootstrap
8. AI Subsystem — LangGraph Multi-Agent Pipeline
9. RAG Subsystem — ASHA Knowledge Chatbot
10. Doctor AI Assistant
11. AI Nutrition Advisor
12. Telegram Bot Integration
13. Flask Blueprints & REST API Reference
14. Dashboard Layer (Jinja2 HTML)
15. Alert & Notification System
16. Document Upload & Analysis Pipeline
17. Risk Scoring Engine
18. Complete User Workflows (Step-by-Step)
19. Screenshot Analysis & UI Description
20. Installation & Setup Guide
21. LangSmith Observability Integration
22. Future Scope & Roadmap
23. Coding Conventions & Patterns

---

1. Project Overview & Problem Statement

1.1 What is MatruRaksha?

MatruRaksha (meaning "Mother's Protection" in Hindi/Sanskrit) is a full-stack, AI-powered maternal healthcare platform engineered for rural and underserved areas of India. It connects three roles — **pregnant mothers**, **ASHA (Accredited Social Health Activist) workers**, and **doctors** — through a unified system combining a Telegram bot, web dashboards, AI decision support, and a RAG-powered chatbot.

1.2 Problem Statement

Rural India faces a maternal health crisis:

- **Limited access to qualified doctors**: PHCs (Primary Health Centres) are often understaffed.
- **Delayed detection of complications**: No systematic monitoring between hospital visits.
- **Communication barriers**: ASHA workers have no real-time support to answer clinical questions.
- **No medical history tracking**: Paper records are unreliable and easily lost.
- **Language barriers**: Most health systems work only in English.
- **No real-time health monitoring**: Mothers have no channel to report symptoms.

1.3 Solution Architecture

```
Mothers (Telegram) ↔ Flask App ↔ AI Pipeline (LangGraph + Groq)
ASHA Workers (Web) ↔ Flask App ↔ RAG Chatbot (ChromaDB + Groq)
Doctors (Web)      ↔ Flask App ↔ AI Assistant (Groq LLaMA)
Admins (Web)       ↔ Flask App ↔ MongoDB Analytics
```

1.4 User Roles

Role	Interface	Primary Responsibility
Mother	Telegram Bot	Self-register, report symptoms, upload documents, receive nutrition advice
ASHA Worker	Web Dashboard	Conduct home visits, log vitals, query RAG chatbot, monitor assigned mothers
Doctor	Web Dashboard	Review AI-flagged cases, provide consultation, chat with AI assistant
Admin	Web Dashboard	Manage users, assign ASHA/Doctor to mothers, view analytics

2. System Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              MatruRaksha System                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────────────┐      │
│  │ Telegram Bot │    │ Web Browser  │    │   Mobile Browser     │      │
│  │ (Mothers)    │    │(ASHA/Doctor/ │    │  (Admin Portal)      │      │
│  │              │    │   Admin)     │    │                      │      │
│  └──────┬───────┘    └──────┬───────┘    └──────────┬───────────┘      │
│         │                   │                       │                  │
│         ▼                   ▼                       ▼                  │
│  ┌──────────────────────────────────────────────────────────────┐      │
│  │                      Flask Application                       │      │
│  │  ┌──────────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐ │      │
│  │  │ /telegram/*  │  │ /admin/* │  │ /asha/*  │  │/doctor/* │ │      │
│  │  │  (webhook)   │  │ (APIs)   │  │ (APIs)   │  │ (APIs)   │ │      │
│  │  └──────────────┘  └──────────┘  └──────────┘  └──────────┘ │      │
│  │  ┌────────────────────┐  ┌──────────────────────────────┐   │      │
│  │  │    /asha/rag/*     │  │       /doctor/ai/*           │   │      │
│  │  │  (RAG Chatbot API) │  │   (AI Assistant API)         │   │      │
│  │  └────────────────────┘  └──────────────────────────────┘   │      │
│  └──────────────────────────────────────────────────────────────┘      │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────┐      │
│  │                         AI Layer                             │      │
│  │  ┌──────────────┐  ┌──────────────┐  ┌────────────────────┐ │      │
│  │  │  LangGraph   │  │  ChromaDB    │  │    Groq API        │ │      │
│  │  │  (6 Agents)  │  │ (Vector DB)  │  │ LLaMA-3.3-70B +    │ │      │
│  │  │  Workflow    │  │              │  │ LLaMA-3.1-8B-inst  │ │      │
│  │  └──────────────┘  └──────────────┘  └────────────────────┘ │      │
│  │  ┌──────────────────────────────────────────────────────┐   │      │
│  │  │  Sentence Transformers (all-MiniLM-L6-v2 embeddings) │   │      │
│  │  └──────────────────────────────────────────────────────┘   │      │
│  └──────────────────────────────────────────────────────────────┘      │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────────────────────────┐       │
│  │   MongoDB        │  │    LangSmith (Observability)         │       │
│  │  (Primary DB)    │  │  Tracing, Debugging, Monitoring      │       │
│  │  Collections:    │  │                                      │       │
│  │  - mothers       │  └──────────────────────────────────────┘       │
│  │  - asha_workers  │                                                 │
```

```
│  │  - doctors       │  ┌──────────────────────────────────────┐  │
│  │  - assessments   │  │    Telegram Bot API (webhook)        │  │
│  │  - consultations │  │  python-telegram-bot v21.0.1         │  │
│  │  - documents     │  └──────────────────────────────────────┘  │
│  │  - messages      │                                            │
│  │  - rag_threads   │                                            │
│  └──────────────────┘                                            │
└──────────────────────────────────────────────────────────────────┘
```

2.2 Request Flow — Assessment Submission

```
ASHA Worker submits form
        │
        ▼
POST /asha/assessment
        │
        ▼
Validate fields (asha_id, mother_id, vitals)
        │
        ▼
assessments_repo.create() → MongoDB
        │
        ▼
prepare_assessment_for_ai() → Build LangGraph state
        │
        ▼
create_matruraksha_graph().invoke() → LangGraph pipeline
        │
     ┌──┴──────────────────────────────┐
     │                                 │
   orchestrator_node            (if AI fails)
     │                                 │
   risk_stratification_node     build_fallback_ai_evaluation()
     │
   symptom_reasoning_node
     │
   trend_analysis_node
     │
   document_analysis_node (if docs uploaded)
     │
   nutrition_lifestyle_node
     │
   communication_node
     │
   finalize_node
        │
```

```
        ▼
build_ai_evaluation() → Transform to MongoDB schema
        |
        ▼
assessments_repo.update_ai_evaluation()
        |
        ▼
send_ai_alerts() → Telegram notifications
        |
        ▼
Return JSON response to ASHA dashboard
```

---

3. Technology Stack & Versions

3.1 Backend

Package	Version	Purpose
`Flask`	3.0.0	Web framework
`Python`	3.10+	Core language
`PyMongo`	4.6.1	MongoDB driver
`python-dotenv`	1.0.0	Environment management
`requests`	2.32.5	HTTP client library

3.2 AI & Machine Learning

Package	Version	Purpose
`groq`	Latest	Groq API client for fast LLM inference
`langchain`	1.2.0	RAG orchestration
`langgraph`	1.0.5	Multi-agent workflow
`langsmith`	0.5.2	AI observability
`langchain-community`	Latest	ChromaDB + embeddings
`chromadb`	0.4.24	Vector database
`sentence-transformers`	2.7.0	Embedding model (`all-MiniLM-L6-v2`)

Package	Version	Purpose
`pydantic`	2.x	Structured output validation

3.3 Communication

Package	Version	Purpose
`python-telegram-bot`	21.0.1	Telegram Bot API
`httpx`	0.27.0	Async HTTP (required for python-telegram-bot)

3.4 Document Processing

Package	Version	Purpose
`pypdf`	3.17.0	PDF text extraction
`pdfplumber`	0.11.0	Advanced PDF parsing
`Pillow`	Latest	Image processing
`pytesseract`	Optional	OCR for document text extraction

3.5 AI Models Used

LLaMA 3.3 70B Versatile (via Groq)

```python
model = "llama-3.3-70b-versatile"
temperature = 0.1   # Very low for clinical accuracy
max_tokens = 4000
response_format = {"type": "json_object"}  # Forces JSON output
```

Used for: Risk stratification agents, document analysis, complex clinical reasoning.

LLaMA 3.1 8B Instant (via Groq)

```python
model = "llama-3.1-8b-instant"
temperature = 0.3   # Slightly higher for conversation
max_tokens = 800-1500
```

Used for: ASHA RAG chatbot, Nutrition advisor, Doctor AI chat, Quick responses.

Sentence Transformers (all-MiniLM-L6-v2)

```python
HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
```

Used for: Creating vector embeddings for ASHA knowledge base documents.

---

4. Directory Structure (Complete)

```
MatruRaksha/
├── .env.example                          # Environment variable template
├── .gitattributes                        # Git configuration
├── README.md                             # Project documentation with screenshots
├── TELEGRAM_REGISTRATION_GUIDE.md       # Telegram bot setup guide
├── requirements.txt                      # Python dependencies (must be created)
├── run.py                                # Flask server entry point (must be
created)
├── run_telegram_bot_new.py               # Telegram bot entry point (must be
created)
|
└── app/                                  # Main application package
    ├── __init__.py                       # App factory (create_app())
    ├── config.py                         # Configuration classes
    ├── db.py                             # MongoDB connection singleton
    |
    ├── ai/                               # AI orchestration layer
    |   ├── __init__.py                   # Exports create_matruraksha_graph()
    |   ├── state.py                      # LangGraph state schema (TypedDict)
    |   ├── agents.py                     # 7 agent node implementations
    |   ├── graph.py                      # LangGraph workflow definition
    |   ├── helpers.py                    # Transform LangGraph → MongoDB schema
    |   ├── fallback.py                   # Rule-based AI fallback (no LangGraph)
    |   ├── alerts.py                     # Telegram alert dispatcher
    |   ├── document_analyzer.py          # Medical document AI analysis
    |   └── nutrition_advisor.py          # AI nutrition recommendations
    |
    ├── rag/                              # RAG subsystem for ASHA chatbot
    |   ├── __init__.py
    |   ├── knowledge_ingestion.py        # PDF → ChromaDB pipeline
    |   ├── retriever.py                  # Vector search + prompt management
```

```
|   ├── safety.py                    # Query safety filters
|   ├── api.py                       # Flask Blueprint: /asha/rag/*
|   └── vector_db/                   # Persisted ChromaDB (pre-built)
|       ├── chroma.sqlite3           # ChromaDB metadata
|       └── f02b2320-.../            # Vector data files
|           ├── data_level0.bin
|           └── header.bin
|
├── doctor/                          # Doctor AI module
|   ├── __init__.py
|   ├── ai_assistant.py              # DoctorAIAssistant class
|   └── ai_api.py                    # Flask Blueprint: /doctor/ai/*
|
├── blueprints/                      # Flask blueprint modules
|   ├── telegram/
|   |   └── routes.py                # /telegram/webhook, /set_webhook
|   ├── admin/
|   |   └── routes.py                # /admin/* APIs
|   ├── admin_dashboard/
|   |   ├── __init__.py
|   |   └── routes.py                # /admin/dashboard/* (HTML pages)
|   ├── asha/
|   |   └── routes.py                # /asha/* APIs
|   ├── asha_dashboard/
|   |   ├── __init__.py
|   |   └── routes.py                # /asha/dashboard/* (HTML pages)
|   ├── doctor/
|   |   └── routes.py                # /doctor/* APIs
|   ├── doctor_dashboard/
|   |   ├── __init__.py
|   |   └── routes.py                # /doctor/dashboard/* (HTML pages)
|   ├── ai/
|   |   └── routes.py                # /ai/* (placeholder)
|   └── api/
|       ├── __init__.py
|       └── routes.py                # /api/* (documents API)
|
├── repositories/                    # Data access layer (must be created)
|   ├── __init__.py
|   ├── mothers_repo.py
|   ├── asha_repo.py
|   ├── doctors_repo.py
|   ├── assessments_repo.py
|   ├── consultations_repo.py
|   ├── documents_repo.py
|   └── messages_repo.py
|
```

```
    ├── services/                         # Service layer (must be created)
    |   ├── __init__.py
    |   ├── telegram_service.py           # Telegram API wrapper
    |   └── telegram_handlers.py          # Message handler logic
    |
    └── templates/                        # Jinja2 HTML templates (must be
created)
        ├── admin/
        |   ├── dashboard.html
        |   ├── mothers.html
        |   ├── asha.html
        |   └── doctors.html
        ├── asha/
        |   ├── dashboard.html
        |   ├── mothers.html
        |   ├── new_assessment.html
        |   ├── stats.html
        |   └── ai_chatbot.html
        └── doctor/
            ├── dashboard.html
            ├── mothers.html
            ├── assessments.html
            ├── consultation_form.html
            ├── consultation_view.html
            └── ai_assistant.html

rag_pdf_source/                           # Source PDFs for RAG (place here)
    ├── ASHA_Module_6_English_2023.pdf
    ├── guidelines-on-asha.pdf
    ├── Notes for ASHA Trainers Part -1 English.pdf
    ├── sba_guidelines_for_skilled_attendance_at_birth.pdf
    └── Guidance_Note-Extended_PMSMA_for_tracking_HRPs.pdf

uploads/                                  # Document uploads directory
    └── documents/
```

---

5. Environment Configuration

5.1 `.env.example` (Complete Reference)

```env
```

```
# ============================================================
# MatruRaksha Environment Configuration
# ============================================================

# Flask Settings
SECRET_KEY=your-secret-key-here-change-in-production
DEBUG=True
DEV_MODE=True

# MongoDB Settings
MONGODB_URI=mongodb://localhost:27017
MONGODB_DB_NAME=matruraksha

# Telegram Bot Settings
# Get from @BotFather on Telegram
TELEGRAM_BOT_TOKEN=your-telegram-bot-token-here

# Groq API Settings (for AI inference)
# Get from https://console.groq.com/
GROQ_API_KEY=your-groq-api-key-here
LLM_MODEL=llama-3.1-70b-versatile

# LangSmith Settings (for AI observability)
# Get from https://smith.langchain.com/
LANGSMITH_API_KEY=your-langsmith-api-key-here
LANGSMITH_PROJECT=matruraksha
LANGCHAIN_TRACING_V2=true

# Server Settings
HOST=0.0.0.0
PORT=8000

# Feature Flags
ENABLE_AI_ADVISORY=True
```

5.2 `app/config.py` (Complete)

```python
```

```python
"""
Configuration Management
Loads environment variables from .env file and provides configuration classes.
"""

import os
from dotenv import load_dotenv

load_dotenv()  # Reads .env file at project root


class Config:
    """Base configuration class. All values from environment variables."""

    # Flask
    SECRET_KEY = os.getenv('SECRET_KEY')
    DEBUG = os.getenv('DEBUG', 'False').lower() == 'true'

    # MongoDB
    MONGODB_URI = os.getenv('MONGODB_URI', 'mongodb://localhost:27017')
    MONGODB_DB_NAME = os.getenv('MONGODB_DB_NAME', 'matruraksha')

    # Telegram
    TELEGRAM_BOT_TOKEN = os.getenv('TELEGRAM_BOT_TOKEN')

    # Groq AI
    GROQ_API_KEY = os.getenv('GROQ_API_KEY')
    LLM_MODEL = os.getenv('LLM_MODEL', 'llama-3.1-70b-versatile')

    # LangSmith
    LANGSMITH_API_KEY = os.getenv('LANGSMITH_API_KEY')
    LANGSMITH_PROJECT = os.getenv('LANGSMITH_PROJECT', 'matruraksha')
    LANGCHAIN_TRACING_V2 = os.getenv('LANGCHAIN_TRACING_V2', 'true')

    # Server
    HOST = os.getenv('HOST', '0.0.0.0')
    PORT = int(os.getenv('PORT', 8000))

    # Feature flags
    DEV_MODE = os.getenv('DEV_MODE', 'False').lower() == 'true'
    ENABLE_AI_ADVISORY = os.getenv('ENABLE_AI_ADVISORY', 'True').lower() == 'true'


class DevelopmentConfig(Config):
    DEBUG = True
```

```python
class ProductionConfig(Config):
    DEBUG = False


config_by_name = {
    'development': DevelopmentConfig,
    'production': ProductionConfig
}


def get_config(config_name='development'):
    return config_by_name.get(config_name, DevelopmentConfig)
```

---

6. Database Schema (MongoDB)

6.1 Collection: `mothers`

```javascript
```

```
{
  _id: ObjectId,

  // Personal Information
  name: String,                          // Full name "Priya Sharma"
  age: Number,                           // 15-50
  phone: String,                         // "+919876543210"
  email: String,                         // Optional

  // Location (two formats — legacy string and structured)
  location: String,                      // "Rampur, Varanasi, UP" (Telegram registe
  address: {                             // Structured (admin registered)
    village: String,
    district: String,
    state: String,
    pincode: String
  },

  // Pregnancy Information
  gestational_age: Number,               // Weeks (Telegram registered)
  current_pregnancy: {                   // (Admin registered)
    gestational_age_weeks: Number,
    edd: Date,                           // Expected Delivery Date (auto-calculated)
    lmp: Date                            // Last Menstrual Period
  },
  edd: String,                           // "2026-05-15" (Telegram registered)

  // Physical data
  weight: Number,                        // kg (optional)
  height: Number,                        // cm (optional)
  blood_group: String,                   // "B+"

  // Telegram Integration
  telegram_chat_id: String,             // "123456789" — Unique Telegram user ID
  telegram_username: String,            // "@username"
  registered_via: String,              // "telegram" | "manual"

  // Assignment
  assigned_asha_id: ObjectId,           // Reference to asha_workers collection
  assigned_doctor_id: ObjectId,         // Reference to doctors collection

  // Status
  status: String,                       // "active" | "inactive"
  risk_level: String,                   // "pending" | "LOW" | "MODERATE" | "HIGH" |
  active: Boolean,                      // true
```

```
  // Risk factors (chronic conditions)
  risk_factors: [String],                // ["hypertension", "diabetes"]
  dietary_restrictions: [String],
  preferred_language: String,            // "hindi"

  // Timestamps
  created_at: Date,
  registered_at: Date,
  updated_at: Date
}
```

6.2 Collection: `asha_workers`

```javascript
{
  _id: ObjectId,
  name: String,                          // "Rekha Devi"
  phone: String,                         // "+919876543211"
  email: String,
  area: String,                          // "Rampur Area"
  district: String,
  state: String,

  // Telegram for notifications
  telegram_chat_id: String,

  // Assignment tracking
  assigned_mothers: [ObjectId],          // Array of mother ObjectIds

  // Performance stats
  total_assessments: Number,
  joined_at: Date,
  created_at: Date,
  updated_at: Date
}
```

6.3 Collection: `doctors`

```javascript
```

```
{
  _id: ObjectId,
  name: String,                          // "Dr. Anjali Singh"
  phone: String,
  email: String,
  specialization: String,               // "Obstetrics & Gynaecology"
  hospital: String,                     // "PHC Varanasi"
  license_number: String,

  // Telegram for notifications
  telegram_chat_id: String,

  // Assignment tracking
  assigned_mothers: [ObjectId],

  // Statistics
  total_consultations: Number,
  high_risk_consultations: Number,

  created_at: Date,
  updated_at: Date
}
```

6.4 Collection: `assessments` ⭐ (Core Collection)

```javascript
```

```
{
  _id: ObjectId,

  // References
  mother_id: ObjectId,                    // → mothers
  asha_id: ObjectId,                      // → asha_workers
  assessment_number: Number,             // Sequential per mother (1, 2, 3...)

  // Assessment Data
  gestational_age_at_assessment: Number, // Gestational week at time of assessment

  vitals: {
    // Blood Pressure
    bp_systolic: Number,                  // e.g., 120
    bp_diastolic: Number,                // e.g., 80

    // Blood markers
    hemoglobin: Number,                   // g/dL — e.g., 11.5
    glucose: Number,                     // mg/dL — random blood glucose

    // Physical
    weight: Number,                      // kg
    heart_rate: Number,                  // bpm (also stored as "pulse")
    pulse: Number,                       // bpm (alternate field name)
    temperature: Number,                // °F or °C (inconsistent — document both)

    // Fetal
    fetal_heart_rate: Number,            // bpm — normal 120-160

    // Additional
    oxygen_saturation: Number,           // % (SpO2)
    spo2: Number                         // Alternate field name
  },

  symptoms: [String],                     // ["headache", "nausea", "swelling"]
  // Valid symptom values:
  // "fever", "headache", "nausea", "vomiting", "bleeding",
  // "discharge", "swelling", "decreased_fetal_movement",
  // "abdominal_pain", "contractions", "blurred_vision",
  // "dizziness", "severe_headache", "vision_changes",
  // "convulsions", "severe_abdominal_pain", "chest_pain",
  // "fainting", "difficulty_breathing", "rash", "jaundice"

  asha_notes: String,                    // Free-text notes from ASHA worker

  documents_uploaded: [ObjectId],        // References to documents collection
```

```
// ==========================================================
// AI EVALUATION (populated after LangGraph runs)
// ==========================================================
ai_evaluation: {
  risk_score: Number,                  // 0-100 (numerical risk score)
  risk_category: String,              // "LOW" | "MODERATE" | "HIGH" | "CRITICAL"
  confidence: Number,                 // 0.0-1.0

  agent_outputs: {
    risk_stratification: {
      risk_level: String,             // "HIGH"
      risk_score: Number,             // 75
      urgency: String,                // "within_24_hours"
      thresholds_exceeded: [String], // ["BP 150/95 > threshold 140/90"]
      clinical_flags: [String],       // ["Hypertension", "Mild anemia"]
      reasoning: String               // 200+ word clinical analysis
    },

    symptom_reasoning: {
      clusters_detected: [String],    // ["Preeclampsia triad"]
      differential_diagnosis: [String],
      urgency_assessment: String,
      severity_assessment: String,
      reasoning: String
    },

    trend_analysis: {
      trend_direction: String,        // "worsening" | "stable" | "improving"
      key_changes: [String],          // ["BP increased 120→150 over 5 days"]
      monitoring_recommendations: [String],
      worsening_indicators: [String],
      stable_indicators: [String],
      reasoning: String
    },

    document_analysis: {
      documents_processed: Number,
      key_findings: [String],
      reasoning: String
    },

    nutrition_lifestyle: {
      dietary_recommendations: [String],
      lifestyle_modifications: [String],
      supplements_needed: [String],
      reasoning: String
```

```
      },

      communication: {
        message_for_mother: String,     // Simple Hindi/English message
        message_for_asha: String,       // Clinical summary
        message_for_doctor: String      // Technical medical summary
      }
    },

    recommended_actions: [String],      // Top 5 action items
    requires_doctor_review: Boolean,    // true if HIGH or CRITICAL
    evaluated_at: Date,
    evaluation_method: String,          // "langgraph" | "rule_based_fallback"
    langsmith_trace_id: String,         // For LangSmith tracing
    agents_invoked: [String],           // ["risk_stratification", "symptom_reasoning
    orchestration_id: String,
    orchestration_timestamp: String
  },

  // Doctor Review Status
  reviewed_by_doctor: Boolean,          // false until doctor reviews
  doctor_reviewed_at: Date,
  consultation_id: ObjectId,            // → consultations

  // Timestamps
  timestamp: Date,                      // When assessment was done
  created_at: Date
}
```

6.5 Collection: `consultations`

```javascript
```

```
{
  _id: ObjectId,

  // References
  assessment_id: ObjectId,              // → assessments
  mother_id: ObjectId,                  // → mothers
  doctor_id: ObjectId,                  // → doctors

  // Clinical Data
  diagnosis: String,                    // "Normal pregnancy, borderline Hb"
  clinical_observations: String,

  // Vitals updated by doctor (if examined in person)
  updated_vitals: {
    bp_systolic: Number,
    bp_diastolic: Number,
    hemoglobin: Number,
    // ... same structure as assessments.vitals
  },

  // Treatment Plan
  treatment_plan: {
    medications: [
      {
        name: String,                   // "Iron + Folic Acid"
        dosage: String,                // "100mg"
        frequency: String,             // "Once daily"
        duration_days: Number
      }
    ],
    nutrition_plan: String,
    activity_restrictions: String,
    follow_up_instructions: String
  },

  // Scheduling
  next_visit_date: Date,

  // AI Override
  overrides_ai_assessment: Boolean,     // true if doctor disagrees with AI
  doctor_risk_assessment: String,       // Doctor's own risk level
  override_reason: String,

  consultation_notes: String,           // Additional free-text

  // Timestamps
```

```
  created_at: Date,
  updated_at: Date
}
```

6.6 Collection: `documents`

```javascript
```

```
{
  _id: ObjectId,

  // References
  mother_id: ObjectId,
  uploaded_by: String,                  // "asha" | "mother" | "doctor"
  uploaded_by_id: ObjectId,

  // Document metadata
  document_type: String,                // "lab_report" | "ultrasound" | "prescriptio
  description: String,

  // File storage
  file_metadata: {
    original_filename: String,
    stored_filename: String,            // "{mother_id}_{timestamp}_{filename}"
    file_path: String,                  // Absolute path on disk
    file_size_bytes: Number,
    file_type: String,                  // ".jpg" | ".pdf" | ".png"
    mime_type: String
  },

  // Text extracted by OCR (pytesseract)
  extracted_text: String,

  // AI Analysis (by document_analyzer.py)
  ai_analysis: {
    key_findings: [String],
    abnormal_values: [String],          // Prefixed with ⚠️
    clinical_summary: String,
    recommendations: [String],
    extracted_text: String
  },

  // Doctor Review
  doctor_review: {
    reviewed_at: Date,
    reviewed_by_doctor_id: ObjectId,
    doctor_name: String,
    notes: String,
    ai_overridden: Boolean,
    corrected_analysis: Object,         // Same structure as ai_analysis
    notification_sent_to: [String]      // ["asha", "mother"]
  },

  // Timestamps
```

```
  uploaded_at: Date,
  created_at: Date
}
```

6.7 Collection: `messages`

```javascript
```

```
{
  _id: ObjectId,

  // References
  mother_id: ObjectId,
  assessment_id: ObjectId,              // Optional
  consultation_id: ObjectId,            // Optional
  document_id: ObjectId,                // Optional

  // Sender
  sender_type: String,                  // "doctor" | "asha" | "ai" | "mother" | "sys
  sender_id: ObjectId,
  sender_name: String,

  // Recipient
  recipient_type: String,               // "mother" | "asha" | "doctor"
  recipient_id: ObjectId,

  // Content
  message_text: String,                 // The message body
  text: String,                         // Alternate field (Telegram handler)

  // Flags
  is_alert: Boolean,                    // true if AI-generated risk alert
  alert_type: String,                   // "ai_risk_high" | "ai_risk_critical"
  related_assessment_id: String,

  from_doctor: Boolean,                 // true for doctor-originated
  from_doctor_id: ObjectId,
  to_asha: Boolean,
  to_asha_id: ObjectId,
  to_mother: Boolean,

  // Delivery
  delivery_status: String,              // "sent" | "failed"
  delivery_error: String,
  telegram_message_id: Number,
  telegram_sent: Boolean,
  telegram_chat_id: String,

  read: Boolean,                        // For notification tracking

  // Timestamps
  timestamp: Date,
```

```
  created_at: Date
}
```

6.8 Collection: `rag_chat_threads`

```javascript
{
  _id: ObjectId,
  asha_id: String,                      // ASHA worker ID (string)
  title: String,                        // Auto-generated from first query

  messages: [
    {
      role: String,                     // "user" | "assistant"
      content: String,                  // For user: question text; for assistant: fu
      confidence: Number,              // 0.0-1.0 (assistant only)
      blocked: Boolean,                // true if safety filter triggered
      timestamp: Date
    }
  ],

  created_at: Date,
  updated_at: Date
}
```

---

7. Application Factory & Bootstrap

7.1 `app/__init__.py` — Application Factory

```python
```

```python
"""
MatruRaksha Flask Application Factory

Implements the Flask app factory pattern.
Creates and configures the Flask application with all blueprints.
"""

from flask import Flask
from app.config import get_config
from app.db import init_db


def create_app(config_name='development'):
    """
    Application factory function.

    Creates and configures the Flask application instance with:
    - Configuration from .env
    - MongoDB connection
    - All blueprints (Telegram, Admin, ASHA, Doctor, AI)

    Args:
        config_name: 'development' or 'production'

    Returns:
        Configured Flask application instance
    """
    app = Flask(__name__)

    # Load configuration from config.py (which reads .env)
    config = get_config(config_name)
    app.config.from_object(config)

    # Initialize MongoDB connection (singleton)
    with app.app_context():
        init_db(app)

    # Register blueprints
    register_blueprints(app)

    # Register error handlers
    register_error_handlers(app)

    return app
```

```python
def register_blueprints(app):
    """Register all Flask blueprints with URL prefixes."""
    from app.blueprints.telegram.routes import telegram_bp
    from app.blueprints.admin.routes import admin_bp
    from app.blueprints.admin_dashboard.routes import admin_dashboard_bp
    from app.blueprints.asha.routes import asha_bp
    from app.blueprints.asha_dashboard.routes import asha_dashboard_bp
    from app.blueprints.doctor.routes import doctor_bp
    from app.blueprints.doctor_dashboard import doctor_dashboard_bp
    from app.blueprints.ai.routes import ai_bp
    from app.blueprints.api.routes import api_bp
    from app.rag.api import asha_rag_bp
    from app.doctor.ai_api import doctor_ai_bp

    app.register_blueprint(telegram_bp, url_prefix='/telegram')
    app.register_blueprint(admin_bp, url_prefix='/admin')
    app.register_blueprint(admin_dashboard_bp)       # prefix: /admin/dashboard
    app.register_blueprint(asha_bp, url_prefix='/asha')
    app.register_blueprint(asha_dashboard_bp)        # prefix: /asha/dashboard
    app.register_blueprint(doctor_bp, url_prefix='/doctor')
    app.register_blueprint(doctor_dashboard_bp)      # prefix: /doctor/dashboard
    app.register_blueprint(ai_bp, url_prefix='/ai')
    app.register_blueprint(api_bp)                   # prefix: /api
    app.register_blueprint(asha_rag_bp)              # prefix: /asha/rag
    app.register_blueprint(doctor_ai_bp)             # prefix: /doctor/ai


def register_error_handlers(app):
    """Register global error handlers."""
    @app.errorhandler(404)
    def not_found(error):
        return {"error": "Resource not found"}, 404

    @app.errorhandler(500)
    def internal_error(error):
        import traceback
        app.logger.error(f"Internal Server Error: {error}")
        app.logger.error(traceback.format_exc())
        return {"error": "Internal server error"}, 500
```

7.2 `app/db.py` — MongoDB Singleton

```python
```

```python
"""
MongoDB Connection Layer

Provides a singleton MongoDB client shared across the entire application.
Uses Flask's application context (g) for thread safety.
"""

from pymongo import MongoClient
from flask import current_app, g

_mongo_client = None  # Global singleton


def init_db(app):
    """Initialize MongoDB connection. Called once during app startup."""
    global _mongo_client

    if _mongo_client is None:
        mongo_uri = app.config['MONGODB_URI']
        _mongo_client = MongoClient(mongo_uri)

        try:
            _mongo_client.admin.command('ping')
            app.logger.info(f"✓ MongoDB connected: {mongo_uri}")
        except Exception as e:
            app.logger.error(f"× MongoDB connection failed: {e}")
            raise


def get_db():
    """
    Get MongoDB database instance.
    Uses Flask's 'g' context for thread safety.
    """
    if 'db' not in g:
        db_name = current_app.config['MONGODB_DB_NAME']
        g.db = _mongo_client[db_name]
    return g.db


def get_collection(collection_name):
    """Convenience function to get a specific collection."""
    db = get_db()
    return db[collection_name]
```

7.3 `run.py` — Server Entry Point (Must Create)

```python
"""Flask server entry point."""

from app import create_app

app = create_app('development')

if __name__ == '__main__':
    host = app.config.get('HOST', '0.0.0.0')
    port = app.config.get('PORT', 8000)
    debug = app.config.get('DEBUG', True)

    print(f"\n🚀 MatruRaksha Server starting on http://{host}:{port}")
    print(f"   Debug mode: {debug}")
    print(f"   Access at: http://localhost:{port}/admin/dashboard/")

    app.run(host=host, port=port, debug=debug)
```

7.4 `requirements.txt` (Must Create)

```txt
```

```
# Web Framework
Flask==3.0.0
python-dotenv==1.0.0
requests==2.32.5

# Database
pymongo==4.6.1

# AI & LLM
groq
langchain==1.2.0
langgraph==1.0.5
langsmith==0.5.2
langchain-community
langchain-classic

# Vector Database
chromadb==0.4.24

# Embeddings
sentence-transformers==2.7.0

# Telegram Bot
python-telegram-bot==21.0.1
httpx==0.27.0

# Document Processing
pypdf==3.17.0
pdfplumber==0.11.0
Pillow
pytesseract

# Data Validation
pydantic>=2.0.0
```

---

8. AI Subsystem — LangGraph Multi-Agent Pipeline

8.1 State Schema (`app/ai/state.py`)

The `MatruRakshaState` TypedDict defines the shared state that flows through all agent nodes in the LangGraph workflow:

```python
```

```python
from typing import TypedDict, List, Dict, Optional, Literal

class MatruRakshaState(TypedDict, total=False):
    # ── INPUT DATA ──────────────────────────────────────────
    assessment_id: str
    mother_id: str
    assessment_type: Literal["routine", "emergency", "follow_up"]
    vitals: Dict                          # Full vitals dict
    symptoms: List[str]                   # List of symptom strings
    risk_factors: List[str]               # Chronic conditions
    gestational_week: int
    has_lab_results: bool
    has_uploaded_documents: bool
    historical_assessments: List[Dict]    # Previous assessments
    mother_profile: Dict                  # Name, age, language, etc.

    # ── AGENT OUTPUTS ───────────────────────────────────────
    risk_stratification_result: Dict
    symptom_reasoning_result: Dict
    trend_analysis_result: Dict
    document_analysis_result: Dict
    nutrition_lifestyle_result: Dict
    communication_result: Dict

    # ── FINALIZE NODE OUTPUT ────────────────────────────────
    orchestration_id: str
    overall_risk_level: Literal["LOW", "MODERATE", "HIGH", "CRITICAL"]
    confidence_score: float
    recommended_actions: List[Dict]
    requires_doctor_review: bool
    agents_invoked: List[str]
    timestamp: str

    # ── FINALIZE EXTRA FIELDS ───────────────────────────────
    final_results: Dict
    workflow_complete: bool
    completed_at: str
    final_risk_level: str
    final_risk_score: int
    final_confidence: float
```

8.2 LangGraph Workflow (`app/ai/graph.py`)

```
START
  |
```

```
  ▼
[orchestrator_node]           → Determines which agents to invoke
  |                             Adds "agents_invoked" list to state
  ▼
[risk_stratification_node]    → Core risk scoring (0-100)
  |                             Uses Groq LLaMA-3.3-70B with JSON output
  |
  ├──(if "symptom_reasoning" in agents_invoked)──►[symptom_reasoning_node]
  |                                                        |
  |   ◄──────────────────────────────────────────────────────
  |
  ├──(if "trend_analysis" in agents_invoked)───►[trend_analysis_node]
  |                                                        |
  |   ◄──────────────────────────────────────────────────────
  |
  ├──(if "document_analysis" in agents_invoked)►[document_analysis_node]
  |                                                        |
  ▼                                                        |
[nutrition_lifestyle_node]    ◄──────────────────────────────
  |
  ▼
[communication_node]
  |
  ▼
[finalize_node]
  |
  ▼
END
```

Creating the graph:

```python
```

```python
def create_matruraksha_graph():
    workflow = StateGraph(MatruRakshaState)

    # Add nodes
    workflow.add_node("orchestrator", orchestrator_node)
    workflow.add_node("risk_stratification", risk_stratification_node)
    workflow.add_node("symptom_reasoning", symptom_reasoning_node)
    workflow.add_node("trend_analysis", trend_analysis_node)
    workflow.add_node("document_analysis", document_analysis_node)
    workflow.add_node("nutrition_lifestyle", nutrition_lifestyle_node)
    workflow.add_node("communication", communication_node)
    workflow.add_node("finalize", finalize_node)

    # Entry point
    workflow.set_entry_point("orchestrator")

    # Edges
    workflow.add_edge("orchestrator", "risk_stratification")
    workflow.add_conditional_edges("risk_stratification", should_run_symptom_reasoni
    workflow.add_conditional_edges("symptom_reasoning", should_run_trend_analysis, {
    workflow.add_conditional_edges("trend_analysis", should_run_document_analysis, {
    workflow.add_edge("document_analysis", "nutrition_lifestyle")
    workflow.add_edge("nutrition_lifestyle", "communication")
    workflow.add_edge("communication", "finalize")
    workflow.add_edge("finalize", END)

    return workflow.compile()
```

8.3 Risk Stratification Agent

System Prompt:

```
You are an expert obstetrician with 25+ years experience.
You MUST follow this STRICT SCORING SYSTEM:

DANGER SYMPTOMS (Each = +40 points, CRITICAL priority):
- bleeding (antepartum hemorrhage)
- decreased_fetal_movement (fetal distress)
- severe_headache (preeclampsia)
- vision_changes (preeclampsia)
- convulsions (eclampsia)
- severe_abdominal_pain (placental abruption)

SEVERE VITAL ABNORMALITIES (Each = +30 points):
- BP ≥ 160/110 mmHg (severe hypertension)
- Hemoglobin < 7 g/dL (severe anemia)
- Temperature > 102°F (high fever)
- Glucose > 200 mg/dL (severe hyperglycemia)

MODERATE ABNORMALITIES (Each = +20 points):
- BP 140-160/90-110 mmHg (hypertension)
- Hemoglobin 7-10 g/dL (moderate anemia)
- Temperature 100.4-102°F (fever)
- Glucose 140-200 mg/dL (hyperglycemia)

RISK LEVELS:
- CRITICAL: 80-100 (immediate emergency)
- HIGH: 60-79 (within 24 hours)
- MODERATE: 30-59 (within week)
- LOW: 0-29 (routine monitoring)
```

Pydantic Output Schema:

```python
class RiskStratificationOutput(BaseModel):
    agent: str = Field(default="risk_stratification")
    risk_level: str = Field(..., description="LOW, MODERATE, HIGH, or CRITICAL")
    confidence: float = Field(..., ge=0.0, le=1.0)
    risk_score: int = Field(..., ge=0, le=100)
    threshold_violations: List[str] = Field(default_factory=list)
    clinical_flags: List[str] = Field(default_factory=list)
    referral_urgency: str = Field(...)   # "immediate" | "within_24_hours" | "within
    reasoning: str = Field(..., min_length=100)
```

API Call:

```python
```

```python
def call_groq_structured(system_msg: str, user_msg: str, temp: float = 0.1) -> str:
    response = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {"role": "system", "content": system_msg},
            {"role": "user", "content": user_msg}
        ],
        temperature=temp,
        max_tokens=4000,
        response_format={"type": "json_object"}  # Forces JSON-only output
    )
    return response.choices[0].message.content
```

Fallback Logic (when Groq API fails):

```python
# Pure rule-based scoring applied if LangGraph call fails
danger_symptoms = ["bleeding", "decreased_fetal_movement", "severe_headache",
                   "vision_changes", "convulsions", "severe_abdominal_pain"]

for sym in symptoms:
    if sym in danger_symptoms:
        risk_score += 40  # +40 per danger symptom

if bp_sys >= 160 or bp_dias >= 110:
    risk_score += 30      # Severe hypertension
elif bp_sys >= 140 or bp_dias >= 90:
    risk_score += 20      # Hypertension

if hb < 7:
    risk_score += 30      # Severe anemia
elif hb < 10:
    risk_score += 20      # Moderate anemia

risk_score = min(risk_score, 100)  # Cap at 100
```

8.4 Symptom Reasoning Agent

Identifies symptom clusters and patterns:

- **Preeclampsia triad:** headache + edema + vision changes + high BP
- **Preterm labor:** contractions + back pain + pelvic pressure
- **Placental abruption:** bleeding + severe abdominal pain + uterine tenderness
- **Infection:** fever + abdominal pain + discharge

```python
class SymptomReasoningOutput(BaseModel):
    agent: str = Field(default="symptom_reasoning")
    symptom_clusters_detected: List[str]
    differential_diagnosis: List[str]
    recommended_questions: List[str]
    urgency_assessment: str              # "immediate" | "urgent" | "within_24_hours
    reasoning: str = Field(..., min_length=100)
```

8.5 Trend Analysis Agent

Compares vitals across historical assessments to detect:

- BP creep (gestational hypertension development)
- Progressive hemoglobin decline (anemia worsening)
- Excessive or insufficient weight gain

```python
class TrendAnalysisOutput(BaseModel):
    agent: str = Field(default="trend_analysis")
    trend_direction: str                 # "improving" | "stable" | "worsening"
    key_changes: List[str]               # "BP increased from 120/80 to 145/90 over
    monitoring_recommendations: List[str]
    reasoning: str = Field(..., min_length=100)
```

8.6 Nutrition & Lifestyle Agent

```python
class NutritionLifestyleOutput(BaseModel):
    agent: str = Field(default="nutrition_lifestyle")
    dietary_recommendations: List[str]   # Iron-rich foods, glucose management
    lifestyle_modifications: List[str]   # Activity, rest, stress
    supplements_needed: List[str]        # IFA, Calcium, etc.
    reasoning: str = Field(..., min_length=100)
```

8.7 Communication Agent

Generates three separate messages in appropriate language/tone:

1. **Mother**: Simple, reassuring, non-medical (Hindi/local language patterns)
2. **ASHA**: Clinical summary with action items
3. **Doctor**: Technical medical summary

```python
class CommunicationOutput(BaseModel):
    agent: str = Field(default="communication")
    message_for_mother: str = Field(..., min_length=50)
    message_for_asha: str = Field(..., min_length=50)
    message_for_doctor: str = Field(..., min_length=50)
    reasoning: str = Field(..., min_length=30)
```

8.8 Finalize Node

Aggregates all agent results:

```python
def finalize_node(state):
    results = {
        "risk_stratification": state.get("risk_stratification_result", {}),
        "symptom_reasoning": state.get("symptom_reasoning_result", {}),
        "trend_analysis": state.get("trend_analysis_result", {}),
        "nutrition_lifestyle": state.get("nutrition_lifestyle_result", {}),
        "communication": state.get("communication_result", {}),
        "document_analysis": state.get("document_analysis_result", {})
    }
    state["final_results"] = results
    state["workflow_complete"] = True
    state["completed_at"] = datetime.utcnow().isoformat()
    state["final_risk_level"] = results["risk_stratification"].get("risk_level", "UN
    state["final_risk_score"] = results["risk_stratification"].get("risk_score", 0)
    state["final_confidence"] = results["risk_stratification"].get("confidence", 0.0
    return state
```

8.9 Preparing Assessment for AI (`app/ai/helpers.py`)

```python
```

```python
def prepare_assessment_for_ai(assessment_data, mother_data, historical_assessments):
    """Transform MongoDB assessment into LangGraph input state."""

    ai_input = {
        "assessment_id": str(assessment_data.get('_id', '')),
        "mother_id": str(mother_data.get('_id', '')),
        "assessment_type": "routine",
        "vitals": assessment_data.get('vitals', {}),
        "symptoms": assessment_data.get('symptoms', []),
        "risk_factors": mother_data.get('risk_factors', []),
        "gestational_week": assessment_data.get('gestational_age_at_assessment', 0),
        "has_lab_results": False,
        "has_uploaded_documents": len(assessment_data.get('documents_uploaded', [])),
        "historical_assessments": [],
        "mother_profile": {
            "name": mother_data.get('name', ''),
            "age": mother_data.get('age', 0),
            "preferred_language": mother_data.get('preferred_language', 'hindi'),
            "dietary_restrictions": mother_data.get('dietary_restrictions', []),
            "region": "rural_india"
        }
    }

    # Add historical data for trend analysis
    ai_input["historical_assessments"] = [
        {
            "date": h.get('timestamp').isoformat() if h.get('timestamp') else '',
            "vitals": h.get('vitals', {})
        }
        for h in historical_assessments[:10]
    ]

    # Auto-detect emergency
    vitals = ai_input["vitals"]
    bp_sys = vitals.get('bp_systolic', 0)
    bp_dias = vitals.get('bp_diastolic', 0)
    if bp_sys >= 160 or bp_dias >= 110 or len(ai_input["symptoms"]) >= 4:
        ai_input["assessment_type"] = "emergency"

    return ai_input
```

8.10 Building MongoDB AI Evaluation from LangGraph Output

```python
```

```python
def build_ai_evaluation(langgraph_result, langsmith_trace_id=None):
    """Transform LangGraph output into MongoDB ai_evaluation schema."""

    risk_strat = langgraph_result.get("risk_stratification_result", {})
    risk_level = risk_strat.get("risk_level", "MODERATE")
    risk_score = risk_strat.get("risk_score", 50)
    confidence = risk_strat.get("confidence", 0.75)

    ai_evaluation = {
        "risk_score": risk_score,
        "risk_category": risk_level,
        "confidence": confidence,
        "agent_outputs": {
            # ... all agent results mapped here
        },
        "recommended_actions": [],
        "requires_doctor_review": risk_level in ["HIGH", "CRITICAL"],
        "evaluated_at": datetime.utcnow()
    }

    # Top 5 recommended actions (from nutrition + urgency agents)
    recommended_actions = []
    if risk_level in ["HIGH", "CRITICAL"]:
        urgency = risk_strat.get("referral_urgency", "within_24_hours")
        if urgency == "immediate":
            recommended_actions.insert(0, "🚨 EMERGENCY: Contact doctor immediately"
        elif urgency == "within_24_hours":
            recommended_actions.insert(0, "⚠️ HIGH PRIORITY: See doctor within 24 ho

    ai_evaluation["recommended_actions"] = recommended_actions[:5]

    return ai_evaluation
```

---

9. RAG Subsystem — ASHA Knowledge Chatbot

9.1 Architecture

```
ASHA Worker Query
      |
      ▼
ASHASafetyFilter.validate_query()
      |
  ┌───┴──────────────────┐
  | BLOCKED              | SAFE
```

```
  | (medication/diagnose) |
  ▼                       ▼
Return blocked_response  ASHARAGRetriever.retrieve_documents()
                              |
                              ▼ (metadata filter: audience=asha)
                         ChromaDB similarity_search(k=4)
                              |
                              ▼
                         format_context() → String context
                              |
                              ▼
                         ASHAPromptManager.create_rag_prompt()
                              |
                              ▼
                         Groq LLaMA-3.1-8B-Instant
                         temperature=0.1, max_tokens=1500
                         response_format=json_object
                              |
                              ▼
                         ResponseValidator.validate_response()
                              |
                              ▼
                         calculate_confidence(docs, response, query)
                              |
                              ▼
                         Return to ASHA Portal
```

9.2 Knowledge Ingestion Pipeline (`app/rag/knowledge_ingestion.py`)

Approved PDF Sources:

```python
```

```
APPROVED_SOURCES = {
    "ASHA_Module_6_English_2023.pdf": {
        "year": "2023", "topic": "general_asha_training",
        "authority": "NHM", "audience": "asha"
    },
    "guidelines-on-asha.pdf": {
        "year": "2023", "topic": "asha_guidelines",
        "authority": "NHM", "audience": "asha"
    },
    "Notes for ASHA Trainers Part -1 English.pdf": {
        "year": "2023", "topic": "asha_trainer_notes",
        "authority": "NHM", "audience": "asha"
    },
    "sba_guidelines_for_skilled_attendance_at_birth.pdf": {
        "year": "2023", "topic": "childbirth_attendance",
        "authority": "NHM", "audience": "asha"
    },
    "Guidance_Note-Extended_PMSMA_for_tracking_HRPs.pdf": {
        "year": "2023", "topic": "high_risk_pregnancy",
        "authority": "NHM", "audience": "asha"
    }
}
```

Chunking Configuration:

```python
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""],
    is_separator_regex=False
)
```

Embedding Model:

```python
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
```

Chunk Validation:

```python
def validate_chunk_quality(chunk) -> bool:
    content = chunk.page_content.strip()
    if len(content) < 50:
        return False  # Too short
    alpha_ratio = sum(c.isalpha() for c in content) / len(content)
    if alpha_ratio < 0.5:
        return False  # Mostly numbers/symbols (table headers)
    return True
```

Creating ChromaDB:

```python
vector_store = Chroma.from_documents(
    documents=all_chunks,
    embedding=embeddings,
    persist_directory="app/rag/vector_db",
    collection_name="asha_knowledge"
)
vector_store.persist()
```

To rebuild the vector DB from scratch:

```bash
python -c "
from app.rag.knowledge_ingestion import ASHAKnowledgeIngestion
ingestion = ASHAKnowledgeIngestion()
ingestion.ingest_all_documents()
"
```

9.3 Retriever (`app/rag/retriever.py`)

```python
# Similarity search with audience filter
docs = self.vector_store.similarity_search(
    query=query,
    k=4,                                    # Max 4 chunks (enforced)
    filter={"audience": "asha"}             # Only ASHA-approved content
)
```

Context Formatting:

```python
```

```python
def format_context(self, documents):
    context_parts = []
    for i, doc in enumerate(documents, 1):
        context_parts.append(
            f"[Document {i}]\n"
            f"Source: {doc.metadata.get('source')}\n"
            f"Topic: {doc.metadata.get('topic')}\n"
            f"Content:\n{doc.page_content.strip()}\n"
        )
    return "\n---\n".join(context_parts)
```

9.4 ASHA System Prompt

```
You are an ASHA protocol guidance assistant for MatruRaksha.

ABSOLUTE RULES:
❌ NEVER diagnose medical conditions
❌ NEVER recommend specific medicines or dosages
❌ NEVER make treatment decisions
❌ NEVER reassure about mother/baby safety
❌ NEVER give advice not found in the documents

✅ ALWAYS provide step-based checklists
✅ ALWAYS specify when to refer to doctor
✅ ALWAYS cite source documents
✅ ALWAYS include safety disclaimer
✅ ALWAYS escalate if uncertain

RESPONSE FORMAT (JSON ONLY):
{
  "guidance": "Clear, actionable protocol guidance",
  "checklist": ["Step 1", "Step 2", "Step 3"],
  "escalation_rule": "Exact condition requiring doctor referral",
  "source_documents": ["Source 1", "Source 2"],
  "disclaimer": "AI-assisted guidance only. Doctor verification required."
}
```

9.5 Safety Filters (`app/rag/safety.py`)

Blocked Query Patterns:

```python
```

```
MEDICATION_PATTERNS = [
    r'\b(medicine|medication|drug|tablet|pill|dose|dosage|mg|ml)\b',
    r'\b(paracetamol|aspirin|antibiotic|injection|vitamin|iron tablet)\b',
    r'\bwhat (medicine|drug|medication) (should|can|to)\b',
    r'\b(give|administer|prescribe) (medicine|drug|medication)\b',
]

DIAGNOSIS_PATTERNS = [
    r'\b(diagnose|diagnosis|what (is|are) (the|this) (disease|condition|problem))\b'
    r'\b(does (she|mother) have|is (this|it) (a|an))\b.*\b(disease|condition|infecti
]

TREATMENT_PATTERNS = [
    r'\b(treatment|treat|cure|therapy)\b',
    r'\b(how to treat|best treatment|treatment (for|of))\b',
]

SAFETY_REASSURANCE_PATTERNS = [
    r'\b(is (it|this|baby|mother) safe|will (baby|mother) be (safe|okay|fine))\b',
    r"\b(don't worry|no need to worry|everything (will be|is) (fine|okay))\b",
]
```

Confidence Scoring:

```python
```

```python
def calculate_confidence(documents, response, query):
    score = 0.0

    # Factor 1: Number of retrieved docs (max 0.3)
    doc_count = len(documents)
    if doc_count >= 4:   score += 0.3
    elif doc_count >= 2: score += 0.2
    elif doc_count >= 1: score += 0.1

    # Factor 2: Query term coverage (max 0.3)
    query_terms = set(query.lower().split()) - STOP_WORDS
    doc_text = ' '.join([doc.page_content.lower() for doc in documents])
    matches = sum(1 for term in query_terms if term in doc_text)
    term_coverage = matches / len(query_terms) if query_terms else 0
    score += min(0.3, term_coverage * 0.3)

    # Factor 3: Response completeness (max 0.25)
    has_guidance = bool(response.get('guidance'))
    has_checklist = bool(response.get('checklist')) and len(response.get('checklist'
    has_escalation = bool(response.get('escalation_rule'))
    score += (has_guidance * 0.1) + (has_checklist * 0.1) + (has_escalation * 0.05)

    # Factor 4: Source citations (max 0.15)
    sources = response.get('source_documents', [])
    if len(sources) >= 2: score += 0.15
    elif len(sources) >= 1: score += 0.1

    return min(1.0, max(0.0, score))
```

9.6 Chat Thread Persistence

RAG conversations are persisted in MongoDB's `rag_chat_threads` collection:

```python
```

```
# Create thread
thread = {
    "asha_id": asha_id,
    "title": "New Chat",
    "messages": [],
    "created_at": datetime.utcnow(),
    "updated_at": datetime.utcnow()
}
result = db.rag_chat_threads.insert_one(thread)

# Add message
db.rag_chat_threads.update_one(
    {"_id": ObjectId(thread_id)},
    {
        "$push": {"messages": {"$each": [user_message, assistant_message]}},
        "$set": {
            "updated_at": datetime.utcnow(),
            "title": user_query[:50] + "..." if len(user_query) > 50 else user_query
        }
    }
)
```

---

10. Doctor AI Assistant

10.1 Overview

The Doctor AI Assistant is a **non-RAG** conversational AI that:

- Provides case summaries from MongoDB data
- Highlights abnormal findings with reference ranges
- Detects trends across assessment history
- Explains AI risk scoring reasoning
- Answers conversational questions about specific patients

10.2 Master System Prompt

```
You are a clinical decision-support assistant.

You must:
- Summarize patient data clearly
- Explain AI reasoning transparently
- Highlight trends and abnormalities
- Avoid diagnosis or treatment advice
```

```
- Use professional medical language
- Defer all decisions to the doctor

ABSOLUTE SAFETY RULES (NON-NEGOTIABLE):
❌ No diagnosis
❌ No medication recommendations or dosages
❌ No treatment plans
❌ No reassurance about outcomes
✅ Explain AI reasoning clearly
✅ Present facts, trends, and flags only
✅ Always defer final decision to doctor

Return ONLY valid JSON:
{
  "case_summary": "Brief factual summary",
  "key_abnormal_findings": ["BP: 150/95 mmHg (Normal: <120/80)"],
  "trend_observations": ["BP increasing from 120/80 to 150/95 over 5 days"],
  "ai_flag_reasoning": "Why AI marked this as high risk",
  "urgency_level": "LOW / MODERATE / HIGH / CRITICAL",
  "doctor_note": "This is AI-assisted screening. Clinical judgment required."
}
```

10.3 Case Data Builder (`_build_case_from_db`)

```python
```

```python
def _build_case_from_db(mother_id):
    """
    Fetches from MongoDB:
    1. Mother profile (name, age, blood_group, edd, lmp)
    2. All assessments sorted by timestamp (most recent first)
    3. Latest vitals + symptoms
    4. Full assessment history (up to 10)
    5. Historical vitals for trend (oldest first for chronological analysis)
    6. AI evaluation from latest assessment
    """
    mother = db.mothers.find_one({"_id": ObjectId(mother_id)})
    all_assessments = list(db.assessments.find(
        {"mother_id": ObjectId(mother_id)}
    ).sort("timestamp", -1))

    case_data = {
        "mother_info": {...},
        "gestational_age": ...,
        "risk_level": ...,
        "current_vitals": {...},           # From latest assessment
        "symptoms": [...],
        "historical_vitals": [...],        # Chronological (oldest first)
        "full_assessment_history": [...],  # Up to 10 assessments
        "latest_ai_evaluation": {...},
        "ai_risk_score": {...}
    }
    return case_data
```

10.4 Chat Mode

The chat endpoint (`/doctor/ai/chat/<mother_id>`) accepts natural language questions and returns **plain conversational text** (not JSON):

```python
# The chat response cleans JSON artifacts:
answer = response.choices[0].message.content
answer = answer.replace("```json", "").replace("```", "").strip()
if answer.startswith("{") and answer.endswith("}"):
    answer = "I apologize, let me rephrase that..."  # Fallback if JSON returned
```

Quick Question Examples Supported:

- "Compare latest vs previous assessment"
- "What are the key concerns?"
- "Analyze the BP trend"

- "Give me a quick summary"
- Custom open-ended questions

---

11. AI Nutrition Advisor

11.1 Time-Aware Meal Recommendations

```python
def get_time_context():
    hour = datetime.now().hour
    if 5 <= hour < 10:
        return {"meal_type": "breakfast", "greeting": "Good morning", ...}
    elif 10 <= hour < 12:
        return {"meal_type": "mid_morning_snack", ...}
    elif 12 <= hour < 15:
        return {"meal_type": "lunch", ...}
    elif 15 <= hour < 17:
        return {"meal_type": "afternoon_snack", ...}
    elif 17 <= hour < 21:
        return {"meal_type": "dinner", ...}
    else:
        return {"meal_type": "night_snack", ...}
```

11.2 Health Context Integration

The advisor pulls:

- Latest vitals (BP, Hb, glucose, weight)
- Current risk level from AI evaluation
- Symptoms list
- Doctor/ASHA messages (last 10)
- Gestational week

11.3 Nutrition Query Detection

```python
```

```python
def is_nutrition_query(text):
    text_lower = text.lower()
    nutrition_keywords = [
        'eat', 'food', 'diet', 'nutrition', 'meal',
        'breakfast', 'lunch', 'dinner', 'snack',
        'hungry', 'appetite', 'recipe',
        'vitamin', 'calcium', 'iron', 'protein',
        'खाना', 'भोजन', 'आहार',     # Hindi keywords
        'kya khau', 'kya khana chahiye'
    ]
    return any(keyword in text_lower for keyword in nutrition_keywords)
```

11.4 Output Format

```python
ai_output = {
    "greeting": "Warm, personalized greeting",
    "meal_suggestions": ["2 whole wheat parathas with ghee", "1 boiled egg (protein)
    "nutrition_focus": "Key nutrients and why they matter",
    "foods_to_avoid": ["Raw/undercooked meat — listeria risk"],
    "general_tips": ["Eat slowly", "Drink water between bites"],
    "doctor_alert": "Alert message if health concern detected, else empty"
}
```

---

12. Telegram Bot Integration

12.1 Architecture

The Telegram bot operates in two modes:

1. **Webhook Mode** (production): Telegram pushes updates to `/telegram/webhook`
2. **Polling Mode** (development): `run_telegram_bot_new.py` uses `Application.run_polling()`

12.2 Registration State Machine

New mothers go through a step-by-step registration:

```
STEP 0: /start received
    ↓ Check if telegram_chat_id exists in mothers collection
    ├── EXISTS → Show main menu (existing user)
    └── NEW → Set state = "awaiting_name"

STEP 1: awaiting_name
    ↓ Validate name (not empty, not a command)
    → Store in session, set state = "awaiting_age"

STEP 2: awaiting_age
    ↓ Validate 15 <= age <= 50
    → Store, set state = "awaiting_phone"

STEP 3: awaiting_phone
    ↓ Validate phone format (+91XXXXXXXXXX)
    → Store, set state = "awaiting_location"

STEP 4: awaiting_location
    ↓ Accept any text (Village, District, State)
    → Store, set state = "awaiting_gestational_week"

STEP 5: awaiting_gestational_week
    ↓ Validate 1 <= week <= 42
    → Calculate EDD = today + (40 - week) * 7 days
    → set state = "awaiting_weight"

STEP 6: awaiting_weight (OPTIONAL)
    ↓ Accept number (kg) or "skip"
    → set state = "awaiting_height"

STEP 7: awaiting_height (OPTIONAL)
    ↓ Accept number (cm) or "skip"
    → set state = "awaiting_email"

STEP 8: awaiting_email (OPTIONAL)
    ↓ Accept email or "skip"
    → set state = "awaiting_confirmation"

STEP 9: Confirmation
    → Show summary with inline keyboard buttons:
       [✅ Yes, Register Me] [❌ No, Start Over]

STEP 10: Registration Complete
    → Insert into mothers collection
```

```
    → registered_via = "telegram"
    → Show success message with /help and /status instructions
```

12.3 Commands

Command	Handler	Description
`/start`	`handle_start_command`	Registration (new) or main menu (existing)
`/status`	`handle_status_command`	Assignment status + risk level
`/help`	`handle_help_command`	Show available commands
`/profile`	`handle_profile_command`	View profile details
`/cancel`	During registration	Restart registration

12.4 Main Menu Buttons (Existing Users)

```
[🩺 Health Summary]    → Fetch latest assessment, show vitals + risk
[📄 Upload Documents]  → Instruct to send photo/PDF
[🚨 Alerts]           → Show critical notifications from messages collection
[👩‍⚕️ Doctor Messages]   → Show messages from assigned doctor and ASHA
[💬 Send Message]      → Store message in messages collection for team
```

12.5 Webhook Endpoint (`/telegram/webhook`)

```python
```

```
@telegram_bp.route('/webhook', methods=['POST'])
def webhook():
    update = request.get_json()

    # Handle callback queries (inline buttons)
    if 'callback_query' in update:
        result = telegram_handlers.handle_callback_query(update['callback_query'])
        return jsonify({"status": "ok"}), 200

    message = update.get('message')
    chat_id = message.get('chat', {}).get('id')
    text = message.get('text', '')

    # Document uploads (photos and files)
    if 'photo' in message:
        telegram_handlers.handle_document_upload(chat_id, message['photo'])
    elif 'document' in message:
        telegram_handlers.handle_document_upload(chat_id, message['document'])

    # Commands
    elif text.startswith('/'):
        command = text.split()[0].lower()
        if command == '/start': telegram_handlers.handle_start_command(chat_id, user
        elif command == '/status': telegram_handlers.handle_status_command(chat_id)
        # ...

    # Regular text
    else:
        telegram_handlers.handle_text_message(chat_id, text)
```

12.6 Setting Webhook

bash

```bash
# Via API endpoint
curl -X POST http://localhost:8000/telegram/set_webhook \
  -H "Content-Type: application/json" \
  -d '{"webhook_url": "https://yourdomain.com/telegram/webhook"}'

# Or directly via Telegram API
curl "https://api.telegram.org/bot{TOKEN}/setWebhook?url=https://yourdomain.com/tele
```

12.7 Mother Database Document (Telegram Registered)

javascript

```
{
  _id: ObjectId(),
  name: "Priya Sharma",
  age: 26,
  phone: "+919876543210",
  location: "Rampur, Varanasi, Uttar Pradesh",
  gestational_age: 20,
  edd: "2026-05-15",           // Calculated: today + (40-20)*7 days
  weight: 58,
  height: 160,
  email: null,                 // Skipped
  telegram_chat_id: "123456789",
  telegram_username: "priya_s",
  registered_via: "telegram",  // Key distinguishing field
  active: true,
  risk_level: "pending",
  assigned_asha_id: null,      // Null until admin assigns
  assigned_doctor_id: null,
  created_at: ISODate("2026-01-01T10:30:00Z")
}
```

---

13. Flask Blueprints & REST API Reference

13.1 URL Routing Map

Prefix	Blueprint	Description
`/telegram/`	`telegram_bp`	Telegram bot webhook
`/admin/`	`admin_bp`	Admin REST APIs
`/admin/dashboard/`	`admin_dashboard_bp`	Admin HTML pages
`/asha/`	`asha_bp`	ASHA REST APIs
`/asha/dashboard/`	`asha_dashboard_bp`	ASHA HTML pages
`/asha/rag/`	`asha_rag_bp`	RAG Chatbot API
`/doctor/`	`doctor_bp`	Doctor REST APIs
`/doctor/dashboard/`	`doctor_dashboard_bp`	Doctor HTML pages
`/doctor/ai/`	`doctor_ai_bp`	Doctor AI Assistant API

Prefix	Blueprint	Description
`/api/`	`api_bp`	Documents API

13.2 Admin API Endpoints (`/admin/*`)

`GET /admin/analytics`

Returns system-wide statistics.

```json
{
  "total_mothers": 150,
  "total_asha": 12,
  "total_doctors": 5,
  "total_assessments": 423,
  "risk_distribution": {
    "LOW": 280,
    "MODERATE": 95,
    "HIGH": 42,
    "CRITICAL": 6
  },
  "risk_trend": [
    {"date": "Mar 21", "low": 12, "moderate": 3, "high": 1, "critical": 0},
    // 7 days of data
  ]
}
```

`GET /admin/mothers`

Returns all active mothers with assigned ASHA/doctor names.

`GET /admin/asha`

Returns all ASHA workers with performance metrics.

`GET /admin/doctors`

Returns all doctors with pending review count.

`POST /admin/assign`

Assigns ASHA and/or doctor to a mother.

```json
```

```
// Request
{
  "mother_id": "ObjectId",
  "asha_id": "ObjectId",
  "doctor_id": "ObjectId"
}

// Response
{
  "status": "success",
  "message": "Assignment updated successfully"
}
```

13.3 ASHA API Endpoints (`/asha/*`)

`GET /asha/mothers?asha_id={id}`

Returns assigned mothers with latest risk info.

`POST /asha/assessment`

Core endpoint — submits a new health assessment and triggers AI pipeline.

```
json
```

```
// Request
{
  "asha_id": "ObjectId",
  "mother_id": "ObjectId",
  "vitals": {
    "bp_systolic": 150,
    "bp_diastolic": 95,
    "heart_rate": 88,
    "temperature": 99.1,
    "weight_kg": 62,
    "glucose_mg_dl": 110,
    "hemoglobin_g_dl": 10.2
  },
  "symptoms": ["headache", "swelling"],
  "asha_notes": "Mother reports mild headache. Swelling in feet noticed.",
  "gestational_age_at_assessment": 28
}

// Response (201)
{
  "status": "success",
  "assessment_id": "...",
  "assessment_number": 4,
  "ai_evaluation_status": "completed",
  "ai_evaluation": {
    "risk_category": "HIGH",
    "risk_score": 65,
    "confidence": 0.92,
    "requires_doctor_review": true,
    "recommended_actions": [
      "⚠️ HIGH PRIORITY: See doctor within 24 hours",
      "Iron-rich foods: spinach, lentils, jaggery",
      "Monitor BP twice daily"
    ],
    "agents_invoked": ["risk_stratification", "symptom_reasoning", "trend_analysis",
  },
  "alerts_sent": true
}
```

`POST /asha/upload-document`

Uploads a medical document with AI analysis.

- Form data: `file`, `mother_id`, `asha_id`, `document_type`, `description`, `analyze_with_ai`
- Saves to `uploads/documents/` directory

- Calls `analyze_medical_document()` if image

`GET /asha/documents/{mother_id}`

Lists all documents for a mother.

`GET /asha/stats?asha_id={id}`

Returns ASHA performance statistics.

`GET /asha/notifications/{asha_id}`

Lists notifications for ASHA worker.

13.4 ASHA RAG API Endpoints (`/asha/rag/*`)

`POST /asha/rag/query`

```json
// Request
{
  "query": "What are danger signs at 28 weeks?",
  "asha_id": "optional_asha_id",
  "mother_id": "optional_mother_id"
}

// Response
{
  "status": "success",
  "response": {
    "guidance": "At 28 weeks, the following danger signs require immediate referral.
    "checklist": ["Step 1: Check BP", "Step 2: Ask about fetal movement"],
    "escalation_rule": "Refer immediately if BP > 140/90 or any danger sign present"
    "source_documents": ["ASHA_Module_6_English_2023", "guidelines-on-asha"],
    "disclaimer": "AI-assisted guidance only. Doctor verification required."
  },
  "confidence": 0.87,
  "flag_for_review": false,
  "blocked": false
}
```

`GET /asha/rag/health` — **Health check**

`GET /asha/rag/stats` — **Knowledge base statistics**

`GET /asha/rag/threads?asha_id={id}` — **List chat threads**

`POST /asha/rag/threads` — **Create new chat thread**

`GET /asha/rag/threads/{thread_id}` — **Get thread with messages**

`POST /asha/rag/threads/{thread_id}/messages` — **Add message + get RAG response**

`DELETE /asha/rag/threads/{thread_id}` — **Delete thread**

13.5 Doctor API Endpoints (`/doctor/*`)

`GET /doctor/mothers?doctor_id={id}`

Returns assigned mothers with latest assessment summary.

`GET /doctor/assessments?mother_id={id}`

Returns full assessment history for a mother.

`GET /doctor/assessment/{assessment_id}`

Returns single assessment with all details + consultation.

`POST /doctor/consultation`

Submits doctor's consultation for an assessment.

```json
// Request
{
  "assessment_id": "ObjectId",
  "doctor_id": "ObjectId",
  "diagnosis": "Gestational hypertension with mild anemia",
  "clinical_observations": "BP elevated at 150/95, Hb 10.2 g/dL",
  "treatment_plan": {
    "medications": [
      {"name": "Labetalol", "dosage": "100mg", "frequency": "BD", "duration_days": 1
    ],
    "nutrition_plan": "Iron-rich diet: spinach, lentils, dates",
    "follow_up_instructions": "Recheck BP in 3 days"
  },
  "next_visit_date": "2026-04-04T10:00:00Z",
  "overrides_ai_assessment": false,
  "doctor_risk_assessment": "HIGH"
}
```

`POST /doctor/message`

Sends direct Telegram message to a mother.

`POST /doctor/review-document`

Submits doctor's review of an uploaded document.

13.6 Doctor AI API Endpoints (`/doctor/ai/*`)

`GET /doctor/ai/analyze-case/{mother_id}`

Returns structured AI analysis of a case.

`POST /doctor/ai/analyze-case`

Analyze from request body or mother_id.

`POST /doctor/ai/chat/{mother_id}`

Chat about a specific patient's case.

```
// Request
{"message": "Compare the last two assessments"}

// Response
{
  "status": "success",
  "response": "Based on the assessments, here's the comparison:\n\nLatest (Apr 1):\n
}
```

13.7 API Documents Endpoint (`/api/*`)

`GET /api/documents/{document_id}`

Returns full document details with AI analysis and doctor review.

---

14. Dashboard Layer (Jinja2 HTML)

14.1 Admin Dashboard Routes

URL	Template	Description
`/admin/dashboard/`	`admin/dashboard.html`	KPIs, risk charts, analytics
`/admin/dashboard/mothers`	`admin/mothers.html`	Mother list + assignment controls
`/admin/dashboard/asha`	`admin/asha.html`	ASHA worker overview
`/admin/dashboard/doctors`	`admin/doctors.html`	Doctor overview

14.2 ASHA Dashboard Routes

URL	Template	Description
`/asha/dashboard/?asha_id={id}`	`asha/dashboard.html`	Stats + recent activity
`/asha/dashboard/mothers?asha_id={id}`	`asha/mothers.html`	Assigned mothers
`/asha/dashboard/new-assessment?asha_id={id}`	`asha/new_assessment.html`	Assessment form
`/asha/dashboard/stats?asha_id={id}`	`asha/stats.html`	Performance stats
`/asha/dashboard/documents?asha_id={id}&mother_id={id}`	`asha/documents.html`	Document viewer

14.3 Doctor Dashboard Routes

URL	Template	Description
`/doctor/dashboard/?doctor_id={id}`	`doctor/dashboard.html`	Overview
`/doctor/dashboard/mothers?doctor_id={id}`	`doctor/mothers.html`	Patient list
`/doctor/dashboard/assessments?doctor_id={id}&mother_id={id}`	`doctor/assessments.html`	Assessment history
`/doctor/dashboard/consultation/new?doctor_id={id}&assessment_id={id}`	`doctor/consultation_form.html`	Submit consultation
`/doctor/dashboard/consultation/view?doctor_id={id}&assessment_id={id}`	`doctor/consultation_view.html`	Read-only view

14.4 No Authentication System

> ⚠️ **IMPORTANT**: The current implementation has **no authentication system**. All dashboards pass `asha_id`, `doctor_id`, or `admin` as URL query parameters. There is no login/logout, no sessions, no JWT. In production, you must implement proper authentication.

---

15. Alert & Notification System

15.1 Alert Routing Logic

```python
def send_ai_alerts(assessment_id, mother_id, ai_evaluation, mother_data, asha_data):
    risk_category = ai_evaluation.get('risk_category', 'MODERATE')
    requires_doctor_review = ai_evaluation.get('requires_doctor_review', False)

    # LOW: Mother only
    # MODERATE: Mother + ASHA
    # HIGH/CRITICAL: Mother + ASHA + Doctor (if requires_doctor_review)

    if risk_category == "LOW":
        send_to_mother(...)
    elif risk_category == "MODERATE":
        send_to_mother(...)
        send_to_asha(...)
    elif risk_category in ["HIGH", "CRITICAL"]:
        send_to_mother(...)
        send_to_asha(...)
        if requires_doctor_review:
            send_to_doctor(...)
```

15.2 Message Templates (Fallback)

Mother (MODERATE risk — Hindi):

```
नमस्ते {name} जी,

आपकी जांच में कुछ बातों पर ध्यान देने की जरूरत है।
कृपया 3-5 दिन में डॉक्टर से मिलें।
आशा बहन आपकी मदद करेंगी।

- MatruRaksha
```

Mother (CRITICAL — Hindi):

```
🚨 {name} जी - तत्काल ध्यान दें

आपकी स्वास्थ्य स्थिति गंभीर है।
कृपया तुरंत अस्पताल जाएं या डॉक्टर को बुलाएं।
आशा बहन और डॉक्टर को सूचित कर दिया गया है।

- MatruRaksha
```

ASHA (HIGH risk):

```
⚠️ URGENT: {mother_name}

Risk: HIGH
Action Required: Doctor visit within 24 hours
Please follow up immediately

- MatruRaksha AI
```

15.3 Telegram Service Implementation

```python
# app/services/telegram_service.py
import requests

TELEGRAM_API_BASE = "https://api.telegram.org/bot{token}"

def send_message(chat_id, text, parse_mode="HTML"):
    """Send message to a Telegram chat."""
    token = os.getenv("TELEGRAM_BOT_TOKEN")
    url = f"https://api.telegram.org/bot{token}/sendMessage"

    response = requests.post(url, json={
        "chat_id": chat_id,
        "text": text,
        "parse_mode": parse_mode
    })
    return response.json()

def set_webhook(webhook_url):
    """Set Telegram webhook URL."""
    token = os.getenv("TELEGRAM_BOT_TOKEN")
    url = f"https://api.telegram.org/bot{token}/setWebhook"
    response = requests.post(url, json={"url": webhook_url})
    return response.json().get('ok', False)
```

---

16. Document Upload & Analysis Pipeline

16.1 Upload Flow

```
ASHA submits document
        |
POST /asha/upload-document
        |
```

```
        ▼
Validate file type (.jpg, .jpeg, .png, .pdf, .gif)
        |
        ▼
secure_filename() + unique timestamp naming
Stored as: {mother_id}_{YYYYMMDD_HHMMSS}_{original_filename}
        |
        ▼
Save to: uploads/documents/ directory
        |
        ▼
Create documents collection record
        |
        ▼ (if analyze_with_ai=true AND image file)
pytesseract.image_to_string() → extracted_text
        |
        ▼
Groq LLaMA-3.3-70B (text only model)
Prompt: document_type-specific analysis prompt
        |
        ▼
Parse JSON response:
{
  "key_findings": [...],
  "abnormal_values": ["⚠️ Hb: 9.2 (Expected: >11, Status: Low)"],
  "clinical_summary": "...",
  "recommendations": [...]
}
        |
        ▼
documents_repo.update_ai_analysis()
        |
        ▼
Return to ASHA portal
```

16.2 Document Type Prompts

Lab Report:

```
Analyze the lab report. Focus on:
- Hemoglobin, Blood Pressure, Glucose, Protein, Blood group, RH factor, TSH
- Mark abnormal values with ⚠️
- Provide clinical interpretation in pregnancy context
```

Ultrasound:

```
Extract: Gestational age, fetal measurements, placenta position, amniotic fluid
Identify any concerning observations.
```

Prescription:

```
List medications with dosages. Note pregnancy safety category.
Flag any contraindications for pregnancy.
```

16.3 Doctor Document Review

```python
POST /doctor/review-document
{
  "document_id": "...",
  "doctor_id": "...",
  "mother_id": "...",
  "notes": "Lab report shows mild anemia. Hb 9.2. Start IFA.",
  "ai_overridden": true,
  "corrected_analysis": {
    "key_findings": ["Hb: 9.2 g/dL (Low)"],
    "clinical_summary": "Mild anemia. IFA therapy recommended."
  },
  "notify_to": ["asha", "mother"]
}
```

---

17. Risk Scoring Engine

17.1 Complete Scoring Algorithm

```
DANGER SYMPTOMS (each adds 40 points):
  - bleeding
  - decreased_fetal_movement
  - severe_headache
  - vision_changes
  - convulsions
  - severe_abdominal_pain

SEVERE VITAL ABNORMALITIES (each adds 30 points):
  - BP systolic ≥ 160 OR diastolic ≥ 110
  - Hemoglobin < 7 g/dL
  - Temperature > 102°F
  - Glucose > 200 mg/dL
```

```
MODERATE VITAL ABNORMALITIES (each adds 20 points):
  - BP systolic 140-160 OR diastolic 90-110
  - Hemoglobin 7-10 g/dL
  - Temperature 100.4-102°F
  - Glucose 140-200 mg/dL

RISK CATEGORIES:
  80-100 → CRITICAL (immediate referral)
  60-79  → HIGH (within 24 hours)
  30-59  → MODERATE (within 1 week)
  0-29   → LOW (routine monitoring)
```

17.2 Rule-Based Fallback (When AI Unavailable)

```python
```

```python
def calculate_risk_score_fallback(vital_signs, symptoms):
    score = 0
    risk_factors = []

    # Blood Pressure (0-40 pts)
    bp_sys = vital_signs.get('bp_systolic', 120)
    bp_dia = vital_signs.get('bp_diastolic', 80)
    if bp_sys >= 160 or bp_dia >= 110:
        score += 40; risk_factors.append("Severe hypertension (BP >= 160/110)")
    elif bp_sys >= 140 or bp_dia >= 90:
        score += 30; risk_factors.append("Hypertension (BP >= 140/90)")
    elif bp_sys >= 130 or bp_dia >= 85:
        score += 15; risk_factors.append("Elevated BP")

    # Hemoglobin (0-20 pts)
    hb = vital_signs.get('hemoglobin', 12)
    if hb < 7:   score += 20; risk_factors.append("Severe anemia (Hb < 7)")
    elif hb < 9: score += 15; risk_factors.append("Moderate anemia (Hb < 9)")
    elif hb < 11:score += 8;  risk_factors.append("Mild anemia (Hb < 11)")

    # Weight (0-15 pts)
    weight = vital_signs.get('weight', 60)
    if weight < 45:  score += 15; risk_factors.append("Underweight (< 45kg)")
    elif weight > 90:score += 10; risk_factors.append("Overweight (> 90kg)")

    # Glucose (0-15 pts)
    glucose = vital_signs.get('glucose', 90)
    if glucose >= 200:  score += 15
    elif glucose >= 140:score += 10
    elif glucose < 60:  score += 12  # Hypoglycemia

    # Symptoms (0-10 pts)
    danger_symptoms = ['bleeding', 'severe_headache', 'vision_problems',
                       'chest_pain', 'seizures', 'blurred_vision']
    for symptom in symptoms:
        if symptom.lower() in danger_symptoms:
            score += 10; break  # Only count once

    risk_score = min(score, 100)

    if risk_score >= 70:   risk_category = "CRITICAL"
    elif risk_score >= 50: risk_category = "HIGH"
    elif risk_score >= 30: risk_category = "MODERATE"
    else:                  risk_category = "LOW"

    return {
```

```
        "risk_score": risk_score,
        "risk_category": risk_category,
        "risk_factors": risk_factors,
        "recommended_actions": [...]
    }
```

---

18. Complete User Workflows (Step-by-Step)

18.1 Phase 0: Mother Self-Registration via Telegram

1. Mother searches for the MatruRaksha bot on Telegram
2. Clicks START → Bot checks if `telegram_chat_id` exists in MongoDB
3. If new: Multi-step registration (Name → Age → Phone → Location → Gestational Week → Weight → Height → Email)
4. All steps validated; optional fields can be skipped
5. Confirmation screen with inline buttons
6. If confirmed: Insert into MongoDB with `registered_via: "telegram"`
7. EDD auto-calculated: `today + (40 - gestational_week) * 7 days`
8. Profile appears immediately in admin portal with 📱 Telegram badge

18.2 Phase 1: Admin Assignment

1. Admin opens `/admin/dashboard/mothers`
2. Sees mother with 📱 badge and "⏳ Pending Assignment" status
3. Clicks "Assign" button → Modal with ASHA and Doctor dropdowns
4. Saves → POST `/admin/assign` → MongoDB updated
5. Mother receives Telegram notification with team details

18.3 Phase 2: ASHA Home Visit & Assessment

1. ASHA opens `/asha/dashboard/?asha_id={id}`
2. Clicks assigned mother → "New Assessment"
3. Fills digital form: BP, Hb, Weight, Pulse, Temperature, Glucose, FHR
4. Checks symptom checkboxes (40+ options)
5. Adds ASHA notes
6. Optionally uploads document (photo of lab report)
7. Submits → POST `/asha/assessment`
8. Backend:

- Creates assessment in MongoDB
- Gets 10 previous assessments for trend analysis
- Builds LangGraph input state
- Runs 6-agent pipeline
- Saves AI evaluation to MongoDB
- Sends Telegram alerts (risk-based routing)

9. Returns risk category + recommended actions to ASHA UI

18.4 Phase 3: ASHA Uses RAG Chatbot

1. ASHA opens AI Assistant tab
2. Types clinical question: "What to do if mother at 28 weeks has BP 150/95?"
3. Safety filter validates query (not medication/diagnosis/treatment)
4. ChromaDB semantic search → Top 4 document chunks
5. Context formatted → Groq LLaMA-3.1-8B-Instant called
6. JSON response returned with: guidance, checklist, escalation_rule, source_documents, disclaimer
7. Confidence scored (0.0-1.0); if < 0.7 → flagged for doctor review
8. Chat thread saved to MongoDB

18.5 Phase 4: Doctor Reviews Case

1. Doctor opens `/doctor/dashboard/?doctor_id={id}`
2. Sees patient list sorted by risk (CRITICAL/HIGH first)
3. Clicks "AI Assistant" for specific patient
4. Clicks "Analyze Case" → GET `/doctor/ai/analyze-case/{mother_id}`
5. Backend builds case_data from MongoDB (all assessments, vitals, etc.)
6. Groq LLaMA-3.3-70B generates structured JSON analysis
7. Doctor sees: Case Summary, Abnormal Findings, Trends, AI Reasoning, Urgency
8. Doctor types questions → POST `/doctor/ai/chat/{mother_id}` → Conversational responses
9. Doctor submits consultation → POST `/doctor/consultation`
10. Assessment marked as reviewed; mother notified via Telegram

18.6 Phase 5: Emergency Scenario

1. Mother texts "I'm having severe bleeding!" to Telegram bot
2. `handle_text_message()` detects emergency keywords

3. Immediate response with emergency instructions + ambulance numbers
4. Alert created with type "EMERGENCY"
5. ASHA and Doctor notified via SMS (if Twilio configured) and dashboard alerts
6. Mother's risk level updated to CRITICAL in MongoDB

---

19. Screenshot Analysis & UI Description

The README contains 26 screenshots from GitHub. Based on the codebase and descriptions, here is a detailed analysis of what each section of the UI looks like:

19.1 Admin Dashboard (Screenshots 1-6)

Screenshot 1 — Admin Main Dashboard:

- Header: "MatruRaksha Admin Portal" with dark sidebar
- KPI cards: Total Mothers, Total ASHA Workers, Total Doctors, Total Assessments
- Risk Distribution Pie Chart showing LOW/MODERATE/HIGH/CRITICAL percentages
- Risk Trend Line Graph showing assessments over 7 days
- Navigation sidebar: Dashboard, Mothers, ASHA Workers, Doctors, Analytics
- Color scheme: Professional blue/white with risk-color indicators (green/yellow/orange/red)

Screenshot 2 — Mothers List:

- Data table with columns: Name, Age, Location, Gestational Week, Risk Level, ASHA Assigned, Doctor Assigned, Actions
- 📱 Telegram badge for self-registered mothers
- Risk level badges: GREEN (LOW), YELLOW (MODERATE), ORANGE (HIGH), RED (CRITICAL)
- "Assign" button for each mother
- Search/filter functionality by risk level

Screenshot 3 — Assignment Modal:

- Modal dialog with mother name at top
- "Assign ASHA Worker" dropdown with available ASHA workers
- "Assign Doctor" dropdown with available doctors
- Save button that triggers POST /admin/assign

Screenshot 4 — ASHA Workers List:

- Table: Name, Phone, Area, Assigned Mothers Count, Total Assessments, High Risk Detected, Performance
- Performance badge: Good/Moderate/Needs Attention

Screenshot 5 — Analytics Charts:

- Multi-line risk trend chart (7-day history)
- Pie chart for risk distribution
- Geographic heat map (if enabled)

Screenshot 6 — Doctor Management:

- Table: Name, Specialization, Hospital, Assigned Mothers, Pending Reviews

19.2 ASHA Worker Dashboard (Screenshots 7-13)

Screenshot 7 — ASHA Dashboard Home:

- Summary cards: Total Mothers, High Risk, Pending Reviews, Recent Assessments
- "My Mothers" list with risk indicators
- Activity feed showing last 5 assessments
- Sidebar: Dashboard, My Mothers, New Assessment, AI Assistant, My Stats

Screenshot 8 — New Assessment Form:

- Patient selector dropdown (assigned mothers only)
- Vital signs section with numeric input fields:
  - BP Systolic / BP Diastolic (mmHg)
  - Hemoglobin (g/dL)
  - Weight (kg)
  - Pulse (bpm)
  - Temperature (°F)
  - Glucose (mg/dL)
  - Fetal Heart Rate (bpm)
- Symptom checklist: 40+ checkboxes in a scrollable grid
- ASHA Notes textarea
- Document upload button
- Submit button

Screenshot 9 — Assessment Result:

- Risk level badge (color-coded)
- AI-generated recommendations list
- "Doctor Review Needed" banner for HIGH/CRITICAL
- Agents invoked list

Screenshot 10 — AI RAG Chatbot:

- Split-panel layout:
  - Left: Chat history sidebar (thread titles)
  - Right: Chat messages area
- Message bubbles (user on right, AI on left)
- Input field + Send button
- Quick question buttons: "Danger Signs", "BP Monitoring", "When to Refer"
- Confidence score shown per AI response (e.g., "87% confidence")
- Disclaimer banner: "AI-assisted guidance only"
- Source citations shown per response

Screenshot 11 — Document Upload View:

- File type selector
- Upload area (drag & drop or click)
- AI Analysis panel showing findings and abnormal values
- Download button for original file

Screenshot 12 — Mother Profile (ASHA View):

- Profile card with photo (initials circle)
- Vital stats summary
- Assessment history timeline
- Document list

Screenshot 13 — ASHA Statistics:

- Performance metrics: Total assessments, Risk breakdown
- Bar charts for weekly activity

19.3 Doctor Dashboard (Screenshots 14-20)

Screenshot 14 — Doctor Main Dashboard:

- Patient list with risk badges

- Pending Reviews count prominently displayed
- Critical cases highlighted in red banner
- Filter by risk level tabs: ALL / LOW / MODERATE / HIGH / CRITICAL

Screenshot 15 — Patient Assessment History:

- Timeline view of all assessments
- Each assessment card shows: Date, Gestational Week, BP, Hb, Risk, Reviewed status
- "New Consultation" button
- "View AI Analysis" button

Screenshot 16 — Doctor AI Assistant Interface:

- Two-panel layout:
  - Left sidebar: Patient list with risk badges
  - Right main area: Chat interface + Analysis panel
- "Analyze Case" button (triggers full AI analysis)
- Chat area with message bubbles:
  - Doctor messages: Purple/right-aligned
  - AI responses: White/left-aligned
- Typing indicator (three animated dots)
- Quick question buttons:
  - 📊 Compare Assessments
  - ⚠️ Key Concerns
  - 📈 BP Trend Analysis
  - 📝 Quick Summary
- Markdown rendering (headers, bold, lists)

Screenshot 17 — AI Case Analysis Panel:

- 📋 Case Summary section
- 🔴 Key Abnormal Findings (formatted as "Parameter: Value (Normal: Range)")
- 📈 Trend Observations
- 🤖 AI Flag Reasoning
- ⚠️ Urgency Level badge

Screenshot 18 — Consultation Form:

- Patient header with name and risk
- Diagnosis textarea

- Clinical observations textarea
- Treatment plan section (medications table + nutrition + activity)
- Follow-up date picker
- "AI Override" toggle (if doctor disagrees with AI risk)
- Submit button

Screenshot 19 — Consultation View (Read-only):

- Shows submitted consultation details
- Doctor name + timestamp
- AI Override indicator

Screenshot 20 — Document Review:

- Document viewer (image or PDF)
- AI analysis panel on right
- Doctor notes input
- "Override AI Analysis" toggle
- "Notify ASHA + Mother" checkboxes

19.4 Telegram Bot Screenshots (Screenshots 21-26)

Screenshot 21 — Registration Start:

- Welcome message with 🌸 emoji
- Step 1: "Please enter your full name:"

Screenshot 22 — Step-by-Step Registration:

- Progressive data collection
- Validation error messages (e.g., "Age must be between 15-50")
- Skip button for optional fields (Weight, Height, Email)

Screenshot 23 — Confirmation Screen:

- Summary of all entered details
- Inline keyboard: [✅ Yes, Register Me] [❌ No, Start Over]

Screenshot 24 — Registration Success:

- 🎉 emoji celebration
- Assignment status: "⏳ Admin will assign your healthcare team soon"

- Available commands listed

Screenshot 25 — Main Menu (Assigned User):

- Inline keyboard with 5 buttons in 2x2 + 1 layout
- Current risk level shown
- Quick access to health summary

Screenshot 26 — Nutrition Advice Response:

- Time-aware greeting (Good morning/afternoon/evening)
- Meal suggestions list
- Nutrition focus explanation
- Foods to avoid
- Tips
- Personalized disclaimer

---

20. Installation & Setup Guide

20.1 Prerequisites

```bash
# Python 3.10+
python --version

# MongoDB 5.0+
mongod --version

# Git
git --version

# Tesseract OCR (optional, for document OCR)
# Ubuntu: sudo apt-get install tesseract-ocr
# macOS: brew install tesseract
# Windows: Download installer from GitHub
```

20.2 Complete Installation

```bash
```

```
# 1. Clone repository
git clone https://github.com/yourusername/MatruRaksha.git
cd MatruRaksha

# 2. Create virtual environment
# Windows
python -m venv env
.\env\Scripts\activate

# macOS/Linux
python3 -m venv env
source env/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Copy environment file
cp .env.example .env

# 5. Edit .env with your values
# - GROQ_API_KEY: Get from https://console.groq.com/ (Free tier available)
# - TELEGRAM_BOT_TOKEN: Create bot via @BotFather on Telegram
# - LANGSMITH_API_KEY: Get from https://smith.langchain.com/
# - MONGODB_URI: Local or Atlas connection string

# 6. Start MongoDB
# Windows: net start MongoDB
# macOS: brew services start mongodb-community
# Linux: sudo systemctl start mongod

# 7. Build RAG Knowledge Base
# Place PDFs in rag_pdf_source/ directory first
mkdir rag_pdf_source
# (Copy the 5 approved PDFs to this directory)

python -c "
from app.rag.knowledge_ingestion import ASHAKnowledgeIngestion
ingestion = ASHAKnowledgeIngestion()
success = ingestion.ingest_all_documents()
print('✅ RAG built!' if success else '❌ RAG failed')
"

# 8. Create initial admin data (optional seed)
python -c "
from app import create_app
from app.db import get_db
```

```
app = create_app()
with app.app_context():
    db = get_db()

    # Create sample ASHA worker
    db.asha_workers.insert_one({
        'name': 'Rekha Devi',
        'phone': '+919876543211',
        'area': 'Rampur',
        'district': 'Varanasi',
        'state': 'UP',
        'assigned_mothers': [],
        'joined_at': __import__('datetime').datetime.utcnow()
    })

    # Create sample doctor
    db.doctors.insert_one({
        'name': 'Dr. Anjali Singh',
        'phone': '+919876543212',
        'specialization': 'Obstetrics & Gynaecology',
        'hospital': 'PHC Varanasi',
        'assigned_mothers': [],
    })

    print('✅ Sample data created')
"

# 9. Run Flask server
python run.py
# Access at: http://localhost:8000

# 10. Run Telegram bot (separate terminal)
source env/bin/activate  # or .\env\Scripts\activate
python run_telegram_bot_new.py
```

20.3 Telegram Bot Setup

```bash
```

```
# Step 1: Create bot via @BotFather
# 1. Open Telegram, search @BotFather
# 2. Send /newbot
# 3. Choose a name (e.g., "MatruRaksha Health Bot")
# 4. Choose a username (e.g., "matruraksha_health_bot")
# 5. Copy the token to .env → TELEGRAM_BOT_TOKEN

# Step 2: Set webhook (for production)
curl -X POST http://localhost:8000/telegram/set_webhook \
  -H "Content-Type: application/json" \
  -d '{"webhook_url": "https://yourdomain.com/telegram/webhook"}'

# For development (polling mode):
python run_telegram_bot_new.py
# This uses python-telegram-bot's Application.run_polling()
```

20.4 `run_telegram_bot_new.py` (Must Create)

```python
```

```python
"""
Standalone Telegram Bot Runner (Polling Mode for Development)
Separate from Flask server.
"""

import asyncio
import os
from dotenv import load_dotenv
from pymongo import MongoClient
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    CallbackQueryHandler, filters
)

load_dotenv()

BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
MONGO_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")
DB_NAME = os.getenv("MONGODB_DB_NAME", "matruraksha")

# Connect to MongoDB
client = MongoClient(MONGO_URI)
db = client[DB_NAME]

print(f"✅ Bot token found: {BOT_TOKEN[:15]}...")
print(f"✅ MongoDB connected")

# User registration state machine (in-memory session)
user_sessions = {}

# Import handlers from app/services/telegram_handlers.py

async def start_command(update, context):
    """Handle /start command."""
    chat_id = str(update.effective_chat.id)
    user = update.effective_user

    # Check if mother already registered
    existing_mother = db.mothers.find_one({"telegram_chat_id": chat_id})

    if existing_mother:
        # Show main menu
        await show_main_menu(update, existing_mother)
    else:
        # Start registration
        user_sessions[chat_id] = {"step": "awaiting_name", "data": {}}
```

```
        await update.message.reply_text(
            "🌸 Welcome to MatruRaksha! 🌸\n\n"
            "I'm here to help you during your pregnancy journey.\n\n"
            "Let's get you registered so our healthcare team can assist you.\n\n"
            "📝 Please enter your full name:"
        )

async def handle_message(update, context):
    """Route incoming text messages."""
    chat_id = str(update.effective_chat.id)
    text = update.message.text
    session = user_sessions.get(chat_id, {})

    if session:
        await handle_registration_step(update, context, chat_id, text, session)
    else:
        # Check if nutrition query
        if is_nutrition_query(text):
            await handle_nutrition_query(update, chat_id)
        else:
            await handle_general_message(update, chat_id, text)

def main():
    """Start the bot."""
    app = Application.builder().token(BOT_TOKEN).build()

    # Handlers
    app.add_handler(CommandHandler("start", start_command))
    app.add_handler(CommandHandler("status", status_command))
    app.add_handler(CommandHandler("help", help_command))
    app.add_handler(CommandHandler("cancel", cancel_command))
    app.add_handler(CallbackQueryHandler(button_callback))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))
    app.add_handler(MessageHandler(filters.PHOTO | filters.Document.ALL, handle_docu

    print("\n🚀 Starting Telegram bot with mother registration...")
    print("Bot is running! Press Ctrl+C to stop.\n")

    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()
```

20.5 Production Deployment

```bash
```

```
# Using gunicorn
gunicorn -w 4 -b 0.0.0.0:8000 "app:create_app('production')"

# Or with uWSGI
uwsgi --http :8000 --module "app:create_app('production')"

# Environment for production
DEBUG=False
DEV_MODE=False
```

---

21. LangSmith Observability Integration

21.1 Configuration

```env
LANGSMITH_API_KEY=lsv2_pt_your_api_key
LANGSMITH_PROJECT=matruraksha
LANGCHAIN_TRACING_V2=true
```

21.2 What LangSmith Tracks

When tracing is enabled in `graph.py`:

```python
if os.getenv("LANGCHAIN_TRACING_V2") == "true":
    os.environ["LANGCHAIN_TRACING_V2"] = "true"
    os.environ["LANGCHAIN_PROJECT"] = os.getenv("LANGSMITH_PROJECT", "matruraksha")
```

LangSmith automatically traces:

- Each LangGraph node execution (timing, tokens used)
- All Groq API calls (prompt, response, latency)
- Token usage and costs
- Error states and retries

21.3 Trace ID Storage

LangSmith trace IDs are stored in MongoDB for debugging:

```python
ai_evaluation["langsmith_trace_id"] = langsmith_trace_id
```

22. Future Scope & Roadmap

22.1 Phase 1: Voice Integration (Q2 2026)

```python
# Planned: Mother sends Hindi voice note
# Whisper API transcribes to text
# Language detection (langdetect)
# Regional TTS for audio responses

# Technologies to integrate:
# - openai-whisper (speech-to-text)
# - langdetect (language detection)
# - TTS libraries for regional languages
```

22.2 Phase 2: Predictive Analytics (Q3 2026)

- ML model (XGBoost/LightGBM) trained on historical data
- Predicts: Preeclampsia risk, Gestational diabetes, Preterm labor probability
- Early intervention 2-4 weeks before clinical presentation

22.3 Phase 3: Government Integration (Q4 2026)

- HMIS (Health Management Information System) API
- MCTS (Mother and Child Tracking System) sync
- Aadhaar verification
- Digital Health ID integration

22.4 Phase 4: Wearables (2027)

- Bluetooth BP monitor → real-time sync
- Fetal Doppler → FHR monitoring
- Glucometer → automated glucose logging

22.5 Phase 5: Multi-Language (2026-2027)

Hindi, Marathi, Tamil, Telugu, Kannada, Odia support for:

- Telegram bot conversations
- Alert messages
- Nutrition recommendations

23. Coding Conventions & Patterns

23.1 Repository Pattern

All database operations go through repository functions, never direct collection access in blueprints:

```python
# ✅ CORRECT
from app.repositories import mothers_repo
mother = mothers_repo.get_by_id(mother_id)

# ❌ WRONG (direct DB access in blueprint)
db = get_db()
mother = db.mothers.find_one({"_id": ObjectId(mother_id)})
```

23.2 Error Handling Pattern

Every blueprint endpoint uses try/except:

```python
try:
    # Business logic
    return jsonify({"status": "success", "data": result}), 200
except Exception as e:
    current_app.logger.error(f"Error: {e}", exc_info=True)
    return jsonify({"error": "Failed", "details": str(e)}), 500
```

23.3 Graceful AI Degradation

AI failure must never block the main workflow:

```python
```

```python
try:
    if AI_AVAILABLE:
        graph = create_matruraksha_graph()
        ai_result = graph.invoke(ai_input)
        ai_evaluation = build_ai_evaluation(ai_result)
    else:
        raise ImportError("LangGraph not available")
except Exception as ai_agent_error:
    # Fallback to rule-based
    ai_evaluation = build_fallback_ai_evaluation(assessment, mother, historical)

# Assessment is ALWAYS saved even if AI fails
assessments_repo.update_ai_evaluation(assessment_id, ai_evaluation)
```

23.4 Field Name Normalization

Multiple field names exist for the same vital (due to schema evolution). Always check both:

```python
# Hemoglobin
hb = vitals.get('hemoglobin') or vitals.get('hemoglobin_g_dl') or 0

# Weight
weight = vitals.get('weight') or vitals.get('weight_kg') or 0

# Glucose
glucose = vitals.get('blood_glucose_random') or vitals.get('glucose_mg_dl') or 0
```

23.5 ObjectId Handling

Always convert to string before JSON serialization:

```python
result = {
    "_id": str(document['_id']),
    "mother_id": str(document['mother_id']),
    # Never: document['_id'] directly (not JSON serializable)
}
```

23.6 Telegram Non-Blocking Alerts

Telegram failures must never block assessment submission:

```python
```

```python
try:
    telegram_service.send_message(chat_id, message)
    result["telegram_sent"] = True
except Exception as e:
    current_app.logger.error(f"Telegram failed (non-blocking): {e}")
    # Do NOT raise — assessment continues
```

23.7 LangGraph State Immutability

LangGraph nodes should always return the complete state with modifications:

```python
def my_node(state: MatruRakshaState) -> MatruRakshaState:
    # Process...
    result = {"key": "value"}
    state["my_result"] = result  # Add to existing state
    return state  # Return full state, not just changes
```

Appendix A: Quick Reference — All API Endpoints

Method	URL	Description
POST	`/telegram/webhook`	Receive Telegram updates
POST	`/telegram/set_webhook`	Configure webhook URL
GET	`/telegram/webhook_info`	Current webhook info
GET	`/telegram/bot_info`	Bot details
GET	`/admin/analytics`	System-wide analytics
GET	`/admin/mothers`	All mothers list
GET	`/admin/asha`	All ASHA workers
GET	`/admin/doctors`	All doctors
POST	`/admin/assign`	Assign ASHA/Doctor to mother
GET	`/asha/mothers`	Mothers assigned to ASHA
POST	`/asha/assessment`	Submit health assessment
GET	`/asha/stats`	ASHA performance stats

Method	URL	Description
POST	`/asha/upload-document`	Upload medical document
GET	`/asha/documents/{mother_id}`	List mother's documents
GET	`/asha/notifications/{asha_id}`	ASHA notifications
POST	`/asha/rag/query`	RAG chatbot query
GET	`/asha/rag/health`	RAG health check
GET	`/asha/rag/threads`	List chat threads
POST	`/asha/rag/threads`	Create chat thread
GET	`/asha/rag/threads/{id}`	Get thread + messages
POST	`/asha/rag/threads/{id}/messages`	Add message, get response
DELETE	`/asha/rag/threads/{id}`	Delete thread
GET	`/doctor/mothers`	Doctor's assigned patients
GET	`/doctor/assessments`	Assessment history
GET	`/doctor/assessment/{id}`	Single assessment detail
POST	`/doctor/consultation`	Submit consultation
POST	`/doctor/message`	Send message to mother
POST	`/doctor/review-document`	Doctor document review
GET	`/doctor/ai/analyze-case/{id}`	AI case analysis
POST	`/doctor/ai/analyze-case`	AI analysis (POST)
POST	`/doctor/ai/chat/{id}`	AI chat about patient
GET	`/doctor/ai/health`	Doctor AI health check
GET	`/api/documents/{id}`	Full document details
GET	`/admin/dashboard/`	Admin HTML page
GET	`/admin/dashboard/mothers`	Admin mothers HTML
GET	`/asha/dashboard/`	ASHA HTML page

Method	URL	Description
GET	`/asha/dashboard/new-assessment`	Assessment form HTML
GET	`/doctor/dashboard/`	Doctor HTML page
GET	`/doctor/dashboard/assessments`	Assessments HTML
GET	`/doctor/dashboard/consultation/new`	Consultation form HTML

Appendix B: MongoDB Collection Index Recommendations

```javascript
// mothers
db.mothers.createIndex({ "telegram_chat_id": 1 }, { unique: true, sparse: true })
db.mothers.createIndex({ "assigned_asha_id": 1 })
db.mothers.createIndex({ "assigned_doctor_id": 1 })
db.mothers.createIndex({ "risk_level": 1 })

// assessments
db.assessments.createIndex({ "mother_id": 1, "timestamp": -1 })
db.assessments.createIndex({ "asha_id": 1, "timestamp": -1 })
db.assessments.createIndex({ "reviewed_by_doctor": 1 })

// rag_chat_threads
db.rag_chat_threads.createIndex({ "asha_id": 1, "updated_at": -1 })

// messages
db.messages.createIndex({ "mother_id": 1, "timestamp": -1 })
db.messages.createIndex({ "to_asha_id": 1, "read": 1 })

// documents
db.documents.createIndex({ "mother_id": 1, "uploaded_at": -1 })
```

Appendix C: Key Constants & Thresholds

```python
```

```
# Risk Score Thresholds
CRITICAL_THRESHOLD = 80
HIGH_THRESHOLD = 60
MODERATE_THRESHOLD = 30

# Vital Normal Ranges
BP_NORMAL_SYSTOLIC_MAX = 120
BP_HYPERTENSION_SYSTOLIC = 140
BP_SEVERE_HYPERTENSION_SYSTOLIC = 160
BP_SEVERE_HYPERTENSION_DIASTOLIC = 110

HEMOGLOBIN_NORMAL_MIN = 11.0  # g/dL
HEMOGLOBIN_MILD_ANEMIA = 10.0
HEMOGLOBIN_MODERATE_ANEMIA = 7.0

GLUCOSE_NORMAL_MAX = 140  # mg/dL random
GLUCOSE_SEVERE = 200

TEMPERATURE_FEVER = 100.4  # °F
TEMPERATURE_HIGH_FEVER = 102.0

# Gestational Age
PRETERM_RISK = 28  # weeks
POSTTERM_RISK = 40  # weeks

# RAG Configuration
MAX_RETRIEVED_CHUNKS = 4
CHUNK_SIZE = 500
CHUNK_OVERLAP = 100
CONFIDENCE_FLAG_THRESHOLD = 0.7

# AI Models
GROQ_MODEL_HEAVY = "llama-3.3-70b-versatile"   # For analysis/reasoning
GROQ_MODEL_FAST = "llama-3.1-8b-instant"        # For chat/real-time

# Embedding Model
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
```

Document generated from complete codebase analysis of MatruRaksha — AI-Supported Rural Maternal Healthcare System

Version: 1.0 | Date: 2026 | Coverage: 100% of project files analyzed